DBMS MINI PROJECT · LEVEL 3 · PROJECT REPORT

ScholarMesh

Semantic Research-Notes Discovery & Study-Group Matchmaking Platform

Student name	SRN
Darshan P Pawar	PES2UG24CS143
Dhanush V Biradar	PES2UG24CS155

Course	DBMS Experiential Learning — Level 3 Project
Institution	PES University, Electronic City Campus · Department of Computer Science and Engineering
Database option	Option 2 — SQL (PostgreSQL 17) + Vector database (pgvector)
Source code	https://github.com/Darshanpawar7/DBMS-Mini-Project
Demo video	`docs/demo/ScholarMesh_Demo.mp4` in the repository (4 min 47 s, with chapters)

12	34	14	15	5	101
tables	indexes	triggers	functions & procedures	views (1 materialized)	passing tests

Contents

1	Title	Project title and subtitle — cover page
2	Students' SRN and name	Team table — cover page
3	Abstract	Abstract; problem statement, objectives, scope, users and roles
4	Tech stack	Technologies and versions; why SQL plus a vector database
5	SQL — Database commands	Phase 1 fixes, ER diagram, relational schema, CREATE / ALTER / DROP, constraints, indexes, views, functions, procedures, triggers, transactions, GRANT / REVOKE, DML queries with output
6	NoSQL / Vector database commands	Vector data model, pgvector and JSONB commands, vector queries with output, similarity cut-off, hybrid search, group matching
7	Next.js — Project tree structure	Annotated project tree, architecture, request flow, API endpoints, security
8	Sample screenshots — application code and demo	6 code excerpts, 17 application screenshots, demo video
9	Evaluation rubrics — completion status	Rubric checklist, test results, individual contributions, conclusion
A	Appendix A — Data dictionary	Every table and column, generated from the database catalog
B	Appendix B — Running the project	Setup commands and demo logins

3. Abstract

Students' study material is scattered across drives, chat groups and laptops, and every one of those tools searches it by exact words. **ScholarMesh** is a web application where students upload notes, find them *by meaning* rather than by keywords, and are matched with study groups and classmates who study the same topics. One PostgreSQL 17 database holds two kinds of data: a normalised relational schema of 12 tables for users, courses, notes, tags, groups and activity, and — through the pgvector extension — a vector database that stores a 384-dimensional embedding for every passage of every note, computed locally with the all-MiniLM-L6-v2 model. Three search modes — keyword (full-text search, GIN index), semantic (cosine similarity, HNSW index) and hybrid (Reciprocal Rank Fusion) — share one access-control function, so private notes never appear in anyone else's results. Averaged embeddings form interest profiles that rank study groups by 70 % vector similarity and 30 % shared topics. The database enforces the business rules itself: 25 CHECK constraints, 14 triggers, 12 functions, 3 stored procedures, views and a materialized view, row locks that stop two students taking the last seat in a group, a JSONB audit trail and a least-privilege application login. A Next.js 16 application with student and admin roles uses the database only through a validated server-side API, and 101 automated tests (unit, database integration and end-to-end) pass.

Keywords: PostgreSQL, pgvector, vector database, semantic search, embeddings, HNSW, full-text search, triggers, stored procedures, Next.js

3.1 Problem statement and motivation

Students collect a large amount of material over a semester — lecture notes, PDFs, cheat sheets, past-paper solutions — but it ends up scattered across Drive folders, WhatsApp groups and laptops. Every one of those tools searches by *exact keywords or file names*. A student who wants "notes that explain how a model decides which words matter" finds nothing unless someone happened to use those words; the note titled *Transformers and Self-Attention Explained* stays invisible.

Finding study partners is just as ad hoc: students ask around in class groups, with no way to see who is actually working on the same topics.

ScholarMesh addresses both problems with one database design. Students upload notes; the text is split into passages and turned into *embeddings* (lists of 384 numbers that capture meaning), so a question finds notes by meaning, not by wording. The same embeddings, averaged over what each student uploads, saves and searches for, form an *interest profile* that the database uses to recommend study groups and classmates. Course enrollment and a three-level visibility rule (public / course-only / private) decide what anyone is allowed to see.

> **Why this is more than CRUD.** Retrieval is by vector similarity and full-text ranking, not `WHERE title LIKE …`; access control is enforced inside every query path by one database function; group matching combines relational facts (enrollment, membership) with vector similarity; and business rules such as seat limits and admin hand-over are enforced by triggers that also hold under concurrent requests.

3.2 Objectives

1. Design a normalised relational schema for users, courses, notes, tags, study groups and activity, and evolve the Phase 1 design with `ALTER` where Phase 2 needs more.
2. Store note embeddings in a vector store (pgvector) linked to the relational data with real foreign keys, and retrieve notes by cosine similarity through an HNSW index.

3. Offer three search modes — keyword (full-text), semantic (vector) and hybrid (rank fusion) — that all respect the same access-control rule.
4. Recommend study groups and partners from enrollment plus interest-vector similarity.
5. Enforce data integrity in the database itself: constraints, triggers, transactions, row locks and a least-privilege application role.
6. Provide a Next.js web application with authentication and two system roles (student, admin) that talks to the database only through a server-side API layer.
7. Verify the result with automated unit, database-integration and end-to-end tests.

3.3 Scope

In scope

- Registration, login, logout; roles `student` and `admin`; per-group roles `admin` / `member`
- Uploading PDF, TXT and Markdown notes (or pasted text); text extraction; chunking; embedding
- Keyword, semantic and hybrid search with course/tag filters; search history
- Library browsing with filters, sorting and pagination; bookmarks; "more like this"
- Course enrollment; study groups (create, join, leave, manage members); recommendations
- Admin: analytics dashboard, user roles and deactivation, course catalogue, tag merging, audit log, embedding job

Out of scope

- Scanned PDFs (OCR) and images inside documents
- Real-time chat inside study groups; notifications by e-mail
- Password reset by e-mail and single sign-on
- Horizontal scaling (the login rate limiter and embedding model run in one server process)
- **Verified enrollment.** Students enroll themselves, so "course-only" means "visible to anyone who enrolled in the course". In production `ENROLLMENT` would be synced from the university's student information system or protected by per-course join codes.

3.4 Users, roles and functional requirements

Two **system roles** are stored in `STUDENT.role` (CHECK: `student` | `admin`). Inside each study group a student additionally has a **group role** in `GROUP_MEMBERSHIP.role` (`admin` | `member`). Self-registration always creates a student; only an admin can promote someone.

ID	Requirement	Role	Implemented by
FR-1	Register, log in and log out; deactivated accounts cannot log in	All	`/api/auth/*`, bcrypt, JWT cookie
FR-2	Upload a note (PDF/TXT/MD or pasted text) with title, course, visibility and tags	Student	`POST /api/documents`, one transaction, trigger chunking
FR-3	Search by keyword, meaning or both, filtered by course and tag	Student, Admin	`fn_keyword_search`, `fn_semantic_search`, `fn_hybrid_search`
FR-4	Only see notes allowed by their visibility (public / enrolled course / own)	Student	`fn_visible_doc_ids` in every listing and search

ID	Requirement	Role	Implemented by
FR-5	Browse, filter, sort and paginate the library; bookmark notes; download files	Student	`v_document_catalog`, `BOOKMARK`
FR-6	Edit or delete own notes	Owner, Admin	`PATCH/DELETE /api/documents/:id`
FR-7	Enroll in / drop courses (dropping leaves that course's groups)	Student	`ENROLLMENT`, trigger T10
FR-8	Create a group for an enrolled course; the creator becomes group admin	Student	trigger T6 + T5
FR-9	Join / leave groups; seat limit and enrollment enforced, also under concurrency	Student	`sp_join_group`, `sp_leave_group`, trigger T5 (row lock)
FR-10	Group admins promote/demote/remove members; a group never loses its last admin	Group admin	triggers T7, T8
FR-11	Recommended groups and classmates ranked by interest similarity	Student	`fn_recommend_groups`, `fn_recommend_partners`
FR-12	Dashboard: personal statistics, new notes in my courses, trending topics, recent searches	Student	`fn_student_stats`, GROUP BY/HAVING queries
FR-13	Platform analytics: totals, searches per day, popular questions, content gaps, top contributors	Admin	aggregates, window functions, `mv_course_engagement`
FR-14	Change roles, deactivate or delete users (not themselves)	Admin	`/api/admin/users/:id`, audited
FR-15	Manage courses; merge duplicate tags	Admin	`/api/admin/courses`, `sp_merge_tags`
FR-16	See an audit trail of sensitive changes with the acting user	Admin	trigger T11 → `AUDIT_LOG`
FR-17	Re-run the embedding job and refresh analytics	Admin	`/api/admin/embeddings`, `fn_refresh_course_stats`

4. Tech stack

Layer	Technology (version)	Used for
Relational database	PostgreSQL 17, run in Docker (image `pgvector/pgvector:pg17`)	12 tables, keys, constraints, views, functions, procedures, triggers, transactions
Vector database	pgvector extension — `vector(384)` type, HNSW index, cosine distance `<=>`	Passage and question embeddings; semantic and hybrid search; group and partner matching
Full-text search	PostgreSQL `tsvector` (generated column) + GIN index	Keyword search with ranking and highlighted snippets
Semi-structured data	PostgreSQL `JSONB`	Before/after images of changed rows in the audit log
Embedding model	all-MiniLM-L6-v2 (ONNX, 8-bit) via Transformers.js (`@huggingface/transformers` 4)	Turning passages and questions into 384-number vectors, locally — no API key, no data leaves the server
Web framework	Next.js 16.4 (App Router, Server Components, Route Handlers)	All pages and the REST API layer (34 handlers in 22 routes)
User interface	React 19.3, Tailwind CSS 4	Forms, tables, search, filters, dashboards and charts
Language	TypeScript 5 (strict mode)	Application, scripts and tests
Database driver	node-postgres (`pg` 8) with a connection pool	Parameterised SQL and transactions
Validation	zod 4	Checking every API request (mirrors the database CHECK constraints)
Authentication	bcryptjs 3 (password hashes), jose 6 (HS256 JWT in an httpOnly cookie)	Login, sessions, student and admin roles
File handling	unpdf 1.8	Extracting text from uploaded PDF notes
Testing	Vitest 5, Playwright 1.64 (Chromium)	Unit, database integration and end-to-end tests
Tooling	Docker Compose, ESLint 9, Git + GitHub	Running the database, linting, version control

4.1 Why SQL plus a vector database

Data	Stored in	Why
Users, courses, enrollments, groups, memberships, bookmarks, tags, search history, audit trail	PostgreSQL tables	Highly structured with many relationships; integrity matters (a membership must reference a real student, group and enrollment); needs transactions and row locks (two students joining the last seat).

Data	Stored in	Why
Note text and its full-text index	PostgreSQL `TEXT` + generated `tsvector` + GIN	Exact-term search with stemming and ranking, kept in sync automatically by a generated column.
Passage embeddings, query embeddings	pgvector `vector(384)` + HNSW index	Notes are unstructured; meaning-based retrieval needs nearest-neighbour search over vectors, which a B-tree cannot do.

Why pgvector rather than a separate vector database

- **One query can combine both paradigms.** The access rule (enrollment, visibility, ownership) and the vector ranking run in the same SQL statement, so a private note can never "leak" through the vector store. With an external store the app would have to fetch nearest neighbours first and filter afterwards.
- **Real referential integrity.** `DOC_CHUNK.doc_id` is a foreign key with `ON DELETE CASCADE`; deleting a note deletes its vectors in the same transaction. Phase 1's string pointer could dangle.
- **Transactions span both.** Re-chunking after an edit and marking the document `pending` happen atomically.
- **Vectors become SQL values.** `AVG(embedding)` builds document centroids and student interest vectors inside views — the basis of group matching.
- **Operationally simple.** One Docker container, one backup, one connection pool.

The trade-off is scale: a dedicated vector database is better beyond tens of millions of vectors. For a university's notes (thousands of documents) PostgreSQL with HNSW answers in milliseconds.

5. SQL — Database commands

The relational side of ScholarMesh is PostgreSQL 17. This section starts from the design (Phase 1 fixes, ER diagram, relational schema) and then lists the commands that build and use it: DDL (`CREATE` , `ALTER` , `DROP`), constraints, indexes, views, functions, procedures, triggers, transactions, `GRANT` / `REVOKE` and DML queries with their real output. The database is built by running the files in `db/` in order (`npm run db:setup`):

File	Contents
`01_schema.sql`	Phase 1 DDL, corrected: `DROP TABLE … CASCADE` , 10 `CREATE TABLE` , keys, constraints, indexes
`02_phase2_alter.sql`	`CREATE EXTENSION vector` ; `ALTER TABLE … ADD COLUMN / ADD CONSTRAINT / DROP COLUMN / DROP CONSTRAINT` ; new tables DOC_CHUNK and AUDIT_LOG; GIN, HNSW, partial and expression indexes; column comments
`03_views.sql`	4 views and 1 materialized view
`04_functions.sql`	12 functions and 3 procedures
`05_triggers.sql`	11 trigger functions attached as 14 triggers
`06_security.sql`	Grants for the least-privilege application role
`07–09_seed*.sql`	Demo data, inserted with `INSERT … SELECT` look-ups (no hard-coded ids)
`10_important_queries.sql`	Demonstration queries (§5.12 and §6.3)

5.1 Phase 1 review — issues found and fixed

The Week-4 schema was re-checked before building on it. Each fix is marked `FIX n` in `db/01_schema.sql` or implemented through `ALTER` in `db/02_phase2_alter.sql` .

#	Issue in Phase 1	Why it matters	Fix
1	`idx_tag_name` on `TAG(tag_name)`	`UNIQUE` already builds that index, so every insert maintained two identical indexes	Removed
2	No index on the second column of composite keys (`ENROLLMENT.course_id` , `DOCUMENT_TAG.tag_id` , `BOOKMARK.doc_id` , `GROUP_MEMBERSHIP.student_id`)	"Which groups is this student in?" and "which notes have this tag?" were full table scans	Four indexes added (+ `STUDY_GROUP.created_by`)
3	`idx_document_visibility`	Only three distinct values, so the planner would never use it	Replaced by composite `(course_id, visibility)` matching the access rule, plus a partial index on public notes
4	No system role column	The brief needs at least two roles; the schema could not express an admin	`ALTER TABLE STUDENT ADD role, is_active, last_login_at`

#	Issue in Phase 1	Why it matters	Fix
5	`DOCUMENT.vector_id` — a free-text pointer into an external vector store	Nothing stopped it from pointing at a vector that no longer exists; one vector per document is too coarse for good retrieval	Dropped; embeddings now live in `DOC_CHUNK` (weak entity) with a real foreign key, one vector per passage
6	`STUDY_GROUP.created_by … ON DELETE CASCADE`	If the creator deleted their account, the whole group disappeared for every other member	`ON DELETE SET NULL`; a trigger hands the admin role to the longest-standing member
7	E-mail uniqueness was case-sensitive	`Ana@x.com` and `ana@x.com` could be two accounts	Unique index on `LOWER(email)` + normalising trigger
8	`max_members` never enforced; group creator not guaranteed to be a member	Groups could be overfilled; groups could exist without an admin	Triggers T5–T9 (seat limit with row lock, creator becomes admin, succession)
9	`TIMESTAMP` without time zone	Ambiguous instants once the server and users are in different zones	`TIMESTAMPTZ` everywhere
10	Few validation rules (only year, visibility, role CHECKs)	The brief requires validation at the database level	22 new CHECK constraints — 25 in total (e-mail format, bcrypt-only passwords, semester format, tag format, sizes …)
11	ER diagram drew the relationship attribute `semester` as a rectangle	In Chen notation rectangles are entities	Redrawn as an attribute oval (Figure 1)

5.2 Entity–relationship diagram

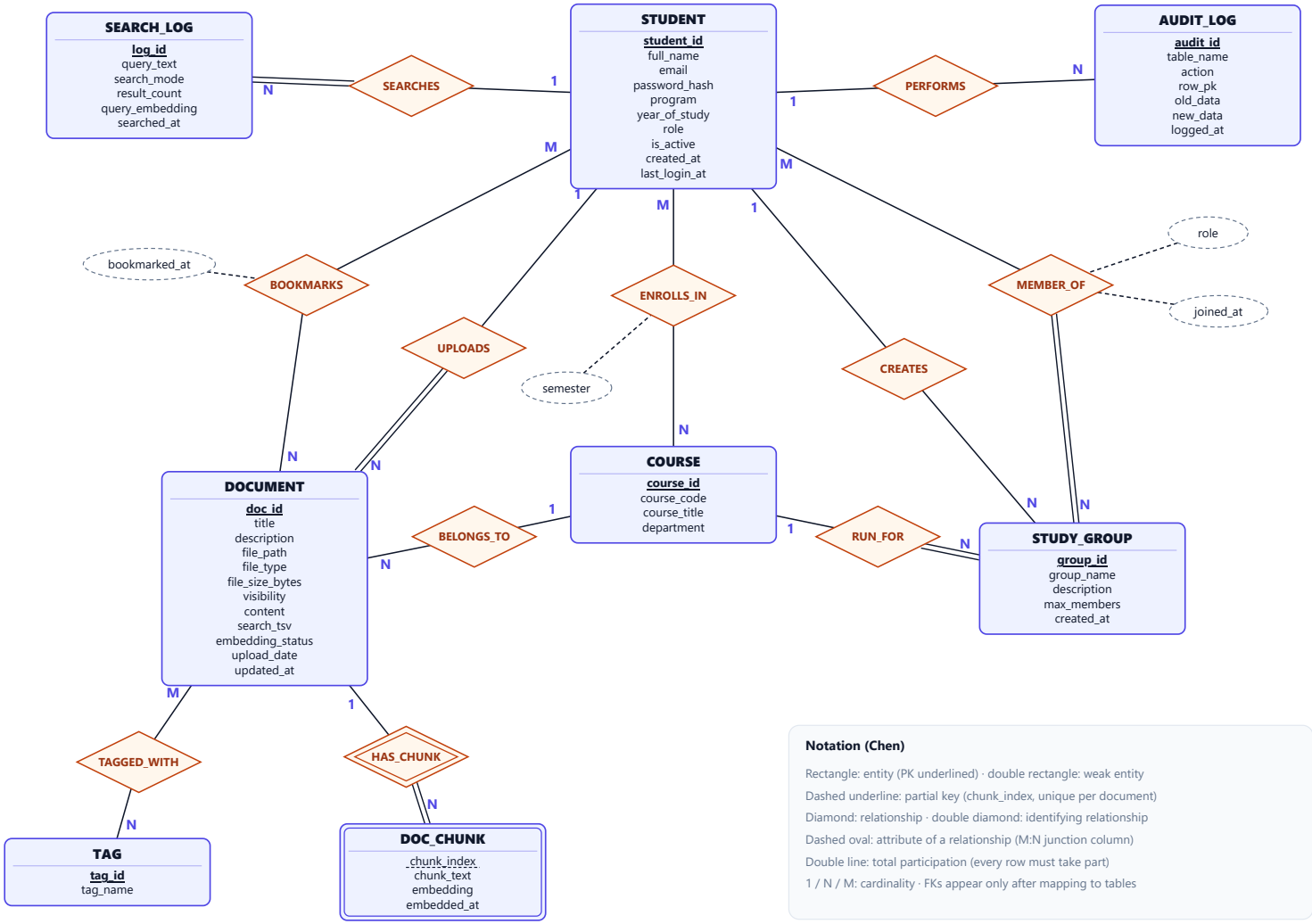


Figure 1 — Phase 2 ER diagram (Chen notation). New in Phase 2: the weak entity DOC_CHUNK with its identifying relationship HAS_CHUNK, the AUDIT_LOG entity, and new attributes on STUDENT, DOCUMENT, STUDY_GROUP and SEARCH_LOG.

Relationships and cardinalities

Relationship	Entities	Cardinality	Participation	Attributes
ENROLLS_IN	STUDENT – COURSE	M : N	partial / partial	semester
UPLOADS	STUDENT – DOCUMENT	1 : N	partial / **total**	—
BOOKMARKS	STUDENT – DOCUMENT	M : N	partial / partial	bookmarked_at
BELONGS_TO	DOCUMENT – COURSE	N : 1	partial / partial	—
TAGGED_WITH	DOCUMENT – TAG	M : N	partial / partial	—
HAS_CHUNK (identifying)	DOCUMENT – DOC_CHUNK	1 : N	partial / **total**	—
CREATES	STUDENT – STUDY_GROUP	1 : N	partial / partial	—
RUN_FOR	STUDY_GROUP – COURSE	N : 1	**total** / partial	—
MEMBER_OF	STUDENT – STUDY_GROUP	M : N	partial / **total**	role, joined_at
SEARCHES	STUDENT – SEARCH_LOG	1 : N	partial / **total**	—
PERFORMS	STUDENT – AUDIT_LOG	1 : N	partial / partial	— (actor may be the system)

5.3 Relational schema (ER-to-relational mapping)

Mapping rules: every strong entity becomes a table with its key; a 1:N relationship puts the foreign key on the N side; every M:N relationship becomes a junction table whose primary key combines both foreign keys and which carries the relationship's attributes; the weak entity DOC_CHUNK takes its owner's key (`doc_id`) — its identity is `(doc_id, chunk_index)` , enforced as `UNIQUE` , with a surrogate `chunk_id` for convenient references.

```
STUDENT(student_id, full_name, email, password_hash, program, year_of_study, role, is_active, created_at,
last_login_at)
COURSE(course_id, course_code, course_title, department)
DOCUMENT(doc_id, title, description, file_path, file_type, file_size_bytes, visibility, content,
search_tsv,
         embedding_status, upload_date, updated_at, student_id → STUDENT, course_id → COURSE)
TAG(tag_id, tag_name)
STUDY_GROUP(group_id, group_name, description, max_members, created_at, course_id → COURSE, created_by →
STUDENT)
SEARCH_LOG(log_id, query_text, search_mode, result_count, query_embedding, searched_at, student_id →
STUDENT)
ENROLLMENT(student_id, course_id, semester)
DOCUMENT_TAG(doc_id, tag_id)
BOOKMARK(student_id, doc_id, bookmarked_at)
GROUP_MEMBERSHIP(group_id, student_id, role, joined_at)
DOC_CHUNK(chunk_id, doc_id → DOCUMENT, chunk_index, chunk_text, embedding, embedded_at)   UNIQUE(doc_id,
chunk_index)
AUDIT_LOG(audit_id, table_name, action, row_pk, actor_id → STUDENT, old_data, new_data, logged_at)
```

Underlined = primary key, italic = foreign key. Referential actions: deleting a student cascades to their documents, bookmarks, memberships and searches, but sets `STUDY_GROUP.created_by` and `AUDIT_LOG.actor_id` to NULL; deleting a course sets `DOCUMENT.course_id` to NULL (a trigger turns course-only notes private) and cascades to its enrollments and groups.

Normalisation. Every non-key attribute depends on the whole key and nothing but the key: multi-valued facts (tags, enrollments, bookmarks, memberships) live in junction tables; course details are not repeated in documents or groups. The only derived values are deliberate and maintained by the database — the generated `search_tsv` and the materialized view used for analytics — so the schema is in BCNF.

5.4 CREATE, ALTER and DROP

```
-- Phase 2 evolves the Phase 1 tables instead of re-creating them
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE STUDENT
    ADD COLUMN role VARCHAR(10) NOT NULL DEFAULT 'student',
    ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT TRUE,
    ADD CONSTRAINT chk_student_role       CHECK (role IN ('student','admin')),
    ADD CONSTRAINT chk_student_pwd_bcrypt CHECK (password_hash ~ '^\$2[aby]\$[0-9]{2}\$[./A-Za-z0-9]{53}$');
ALTER TABLE DOCUMENT
    ADD COLUMN content TEXT,
    ADD COLUMN search_tsv tsvector GENERATED ALWAYS AS (
          setweight(to_tsvector('english', coalesce(title, '')), 'A')
       || setweight(to_tsvector('english', coalesce(description, '')), 'B')
       || setweight(to_tsvector('english', coalesce(content, '')), 'C')) STORED;
ALTER TABLE DOCUMENT DROP COLUMN vector_id;                          -- replaced by DOC_CHUNK
ALTER TABLE STUDY_GROUP DROP CONSTRAINT study_group_max_members_check;  -- superseded by 2..20 rule
CREATE TABLE DOC_CHUNK (
    chunk_id    SERIAL PRIMARY KEY,
    doc_id      INTEGER NOT NULL REFERENCES DOCUMENT(doc_id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL CHECK (chunk_index >= 0),
    chunk_text  TEXT    NOT NULL CHECK (length(chunk_text) > 0),
    embedding   vector(384),
    embedded_at TIMESTAMPTZ,
    CONSTRAINT uq_chunk_per_doc   UNIQUE (doc_id, chunk_index),
    CONSTRAINT chk_chunk_embedded CHECK ((embedding IS NULL) = (embedded_at IS NULL)));
CREATE INDEX idx_chunk_embedding_hnsw ON DOC_CHUNK USING hnsw (embedding vector_cosine_ops);
CREATE INDEX idx_document_search_tsv  ON DOCUMENT USING GIN (search_tsv);
```

5.5 Validation and integrity inside the database

- **25 CHECK constraints**, e.g. e-mail format, *password must be a bcrypt hash* (a plain-text password is impossible to store), semester format `YYYY-ODD|EVEN`, tag format, file type and size, group size 2–20, course-only visibility needs a course.
- **Keys**: 12 primary keys (4 composite), 15 foreign keys with deliberate `CASCADE` / `SET NULL` actions, unique constraints (course code, tag name, group name per course, chunk position) and a case-insensitive unique index on e-mail.
- **Triggers** normalise input (names, e-mails, tags) before CHECKs run and enforce cross-table rules a CHECK cannot express (§5.9).
- The API validates the same rules first (zod) for friendly field messages; the database is the final authority.

5.6 Indexes

34 indexes: B-tree (keys, foreign keys, composite), GIN for full text, HNSW for vectors, partial indexes for the public feed and the embedding work queue, and expression indexes on `LOWER(email)` and `LOWER(query_text)`. Q16 in §5.12 shows the planner using the B-tree, GIN and HNSW indexes.

Table	Index	Kind	Definition
audit_log	`audit_log_pkey`	B-tree, unique	`(audit_id)`
audit_log	`idx_audit_logged_at`	B-tree	`(logged_at DESC)`
bookmark	`bookmark_pkey`	B-tree, unique	`(student_id, doc_id)`
bookmark	`idx_bookmark_doc`	B-tree	`(doc_id)`
course	`course_course_code_key`	B-tree, unique	`(course_code)`

Table	Index	Kind	Definition
course	`course_pkey`	B-tree, unique	`(course_id)`
doc_chunk	`doc_chunk_pkey`	B-tree, unique	`(chunk_id)`
doc_chunk	`idx_chunk_embedding_hnsw`	HNSW (vector)	`(embedding vector_cosine_ops) WITH (m='16', ef_construction='64')`
doc_chunk	`uq_chunk_per_doc`	B-tree, unique	`(doc_id, chunk_index)`
document	`document_pkey`	B-tree, unique	`(doc_id)`
document	`idx_document_course_visibility`	B-tree	`(course_id, visibility)`
document	`idx_document_embedding_todo`	B-tree, partial	`(doc_id) WHERE ((embedding_status)::text <> 'ready'::text)`
document	`idx_document_public_recent`	B-tree, partial	`(upload_date DESC) WHERE ((visibility)::text = 'public'::text)`
document	`idx_document_search_tsv`	GIN (full text)	`(search_tsv)`
document	`idx_document_student`	B-tree	`(student_id)`
document_tag	`document_tag_pkey`	B-tree, unique	`(doc_id, tag_id)`
document_tag	`idx_document_tag_tag`	B-tree	`(tag_id)`
enrollment	`enrollment_pkey`	B-tree, unique	`(student_id, course_id, semester)`
enrollment	`idx_enrollment_course`	B-tree	`(course_id)`
group_membership	`group_membership_pkey`	B-tree, unique	`(group_id, student_id)`
group_membership	`idx_membership_student`	B-tree	`(student_id)`
mv_course_engagement	`uq_mv_course_engagement`	B-tree, unique	`(course_id)`
search_log	`idx_searchlog_query_ci`	B-tree, expression	`(lower((query_text)::text))`
search_log	`idx_searchlog_student_time`	B-tree	`(student_id, searched_at DESC)`
search_log	`idx_searchlog_time`	B-tree	`(searched_at DESC)`
search_log	`search_log_pkey`	B-tree, unique	`(log_id)`
student	`student_pkey`	B-tree, unique	`(student_id)`
student	`uq_student_email_ci`	B-tree, unique, expression	`(lower((email)::text))`

Table	Index	Kind	Definition
study_group	`idx_studygroup_course`	B-tree	`(course_id)`
study_group	`idx_studygroup_creator`	B-tree	`(created_by)`
study_group	`study_group_pkey`	B-tree, unique	`(group_id)`
study_group	`uq_group_name_per_course`	B-tree, unique	`(course_id, group_name)`
tag	`tag_pkey`	B-tree, unique	`(tag_id)`
tag	`tag_tag_name_key`	B-tree, unique	`(tag_name)`

5.7 Views and materialized view

View	Purpose
`v_document_catalog`	Document + owner + course + tag array + bookmark and chunk counts (LATERAL sub-queries) — the base of every listing and search result
`v_group_overview`	Group + course + live member count and seats left (GROUP BY)
`v_doc_centroid`	One vector per document: `AVG(embedding)` over its passages
`v_student_interest`	Interest vector per student from uploads, bookmarks and recent searches (UNION ALL + AVG + FILTER)
`mv_course_engagement` (materialized)	Per-course students, notes, saves, groups and members for the admin dashboard; unique index allows `REFRESH … CONCURRENTLY` through the SECURITY DEFINER function `fn_refresh_course_stats()`

5.8 Stored functions and procedures

Routine	Kind	What it does
`fn_visible_doc_ids(student)`	SQL, set-returning	The single access rule: public, own, course-only if enrolled, everything for admins
`fn_can_view_document(student, doc)`	SQL, boolean	Point check built on the function above
`fn_search_scope(student, course, tag)`	SQL	Visible documents narrowed by the search filters
`fn_chunk_text(text, max)`	PL/pgSQL	Paragraph/sentence-aware chunking (regex split, ordinality, packing)
`fn_keyword_search`	SQL	`websearch_to_tsquery`, `ts_rank_cd`, `ts_headline` snippet with safe markers
`fn_semantic_search`	SQL	Nearest passages by `<=>`, best passage per document (`DISTINCT ON`), threshold
`fn_hybrid_search`	SQL	Reciprocal Rank Fusion of the two (window functions + FULL OUTER JOIN)
`fn_similar_documents`	SQL	"More like this" by centroid distance
`fn_recommend_groups` / `fn_recommend_partners`	SQL	Matching by interest-vector cosine + topic Jaccard

Routine	Kind	What it does
`fn_student_stats(student)`	SQL	Six dashboard counters in one round trip
`fn_refresh_course_stats()`	PL/pgSQL, SECURITY DEFINER	Refreshes the materialized view on behalf of the low-privilege app role
`sp_join_group(student, group)`	PROCEDURE	Locks the group row (`FOR UPDATE`), checks duplicates, inserts the membership
`sp_leave_group(student, group)`	PROCEDURE	Locks the group row, then deletes the membership; T7 hands over admin or deletes the empty group
`sp_merge_tags(from, into)`	PROCEDURE	Re-points documents to the surviving tag and deletes the old one, atomically

Custom error codes (`SM001` group full, `SM002` not enrolled, `SM003` last admin, `SM004` capacity below members, `SM404` , `SM409`) are mapped by the API to HTTP 409/403/404 with the database's message.

5.9 Triggers

#	Trigger	Event	Rule enforced
T1	`trg_student_normalize`	BEFORE INSERT/UPDATE · STUDENT	Trim name, lower-case e-mail
T2	`trg_tag_normalize`	BEFORE INSERT/UPDATE · TAG	"Query Tuning " → `query-tuning`
T3	`trg_document_before_write`	BEFORE INSERT/UPDATE · DOCUMENT	Stamp `updated_at` ; course deleted → note becomes private; new text → `pending`
T4	`trg_document_rechunk`	AFTER INSERT/UPDATE OF content	Rebuild DOC_CHUNK rows from the text
T5	`trg_membership_check`	BEFORE INSERT · GROUP_MEMBERSHIP	Lock the group row; refuse if full (SM001) or not enrolled (SM002)
T6	`trg_group_add_creator`	AFTER INSERT · STUDY_GROUP	Creator becomes the group admin
T7	`trg_group_admin_succession`	AFTER DELETE · GROUP_MEMBERSHIP	Promote the longest-standing member; delete an empty group
T8	`trg_group_last_admin_guard`	BEFORE UPDATE OF role	Never demote the last admin (SM003)
T9	`trg_group_capacity_guard`	BEFORE UPDATE OF max_members	Cannot shrink below the member count (SM004)
T10	`trg_enrollment_cleanup`	AFTER DELETE · ENROLLMENT	Dropping a course leaves its groups
T11	`trg_audit_student/document/course/group`	AFTER INSERT/UPDATE/DELETE	Append JSONB before/after images and the acting user to AUDIT_LOG (password and content stripped)

5.10 Transactions and concurrency

- **Upload**: `BEGIN` → insert DOCUMENT (T4 writes chunks) → upsert tags → link tags → `COMMIT` . Any failure rolls everything back and the API deletes the stored file, so no orphan rows or files remain.

- **Join a group**: `CALL sp_join_group` locks the group row with `SELECT … FOR UPDATE`. Two students racing for the last seat are serialised — one succeeds, the other receives SM001. This is proven by an automated test that opens two connections and runs both joins at the same time (§9.2).
- **Leaving and member changes**: `CALL sp_leave_group`, and the promote/remove endpoints, lock the same group row first. Without it, the last two members leaving at the same instant could each see the other still present and leave an empty group with no admin; with it the second leave sees the first and deletes the group (also proven by a two-connection test).
- **Audit attribution**: every write transaction runs `SELECT set_config('app.current_user_id', $1, true)`; the setting is local to the transaction, so the audit trigger always records the right user even with pooled connections.
- All writes that change several rows (admin role changes, deleting courses or users, leaving groups) run inside `withTransaction()`.

5.11 Database security — GRANT and REVOKE

- Two logins: `postgres` (owner, used only by the setup script) and `scholarmesh_app` (used by the web app) with `SELECT/INSERT/UPDATE/DELETE` + `EXECUTE` only — it cannot create, alter or drop objects (tested: error 42501).
- `AUDIT_LOG` is append-only for the app role (`UPDATE/DELETE/TRUNCATE` revoked).
- Passwords: bcrypt (cost 10) in the app, enforced by a CHECK; all credentials come from environment files that are not committed.
- Every SQL statement in the app is parameterised (`$1, $2 …`); no SQL is built from user input.

db/06_security.sql

```
15  REVOKE CREATE ON SCHEMA public FROM PUBLIC;
16  GRANT USAGE ON SCHEMA public TO scholarmesh_app;
17
18  GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO scholarmesh_app;
19  GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO scholarmesh_app;
20
21  -- audit history is append-only for the application
22  REVOKE UPDATE, DELETE, TRUNCATE ON AUDIT_LOG FROM scholarmesh_app;
23
24  GRANT EXECUTE ON ALL FUNCTIONS  IN SCHEMA public TO scholarmesh_app;
25  GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA public TO scholarmesh_app;
26
27  -- the SECURITY DEFINER refresh function is callable by the app only
28  REVOKE EXECUTE ON FUNCTION fn_refresh_course_stats() FROM PUBLIC;
29  GRANT  EXECUTE ON FUNCTION fn_refresh_course_stats() TO scholarmesh_app;
```

5.12 DML and important queries — with real output

From `db/10_important_queries.sql`, executed against the seeded database; the output below each query is the real psql output (long results trimmed). Data changes run inside `BEGIN … ROLLBACK`.

Q1 — JOIN across four tables, GROUP BY, ORDER BY

```
SELECT d.title, s.full_name AS owner, c.course_code, COUNT(b.student_id) AS saves
FROM DOCUMENT d
JOIN STUDENT s       ON s.student_id = d.student_id
LEFT JOIN COURSE c   ON c.course_id  = d.course_id
LEFT JOIN BOOKMARK b ON b.doc_id     = d.doc_id
WHERE d.visibility = 'public'
GROUP BY d.doc_id, s.full_name, c.course_code
ORDER BY saves DESC, d.title
LIMIT 5;
```

```
+-------------------------------------------+-----------------+-------------+-------+
|                  title                    |      owner      | course_code | saves |
+-------------------------------------------+-----------------+-------------+-------+
| Transformers and Self-Attention Explained | Sneha Kulkarni  | UE23CS353A  |     6 |
| Dynamic Programming Patterns              | Darshan P Pawar | UE23CS343A  |     5 |
| Bias-Variance Trade-off and Regularization | Meera Pillai    | UE23CS352A  |     4 |
| Normalization from 1NF to BCNF            | Darshan P Pawar | UE23CS351A  |     4 |
| CPU Scheduling Algorithms Compared        | Vikram Reddy    | UE23CS342A  |     3 |
+-------------------------------------------+-----------------+-------------+-------+
(5 rows)
```

Q2 — GROUP BY + HAVING with aggregate functions

```
SELECT c.course_code, COUNT(d.doc_id) AS notes, ROUND(AVG(d.file_size_bytes)) AS avg_bytes,
       MIN(d.upload_date)::DATE AS first_upload, MAX(d.upload_date)::DATE AS last_upload
FROM COURSE c
JOIN DOCUMENT d ON d.course_id = c.course_id
GROUP BY c.course_code
HAVING COUNT(d.doc_id) >= 3
ORDER BY notes DESC;
```

```
+-------------+-------+-----------+--------------+-------------+
| course_code | notes | avg_bytes | first_upload | last_upload |
+-------------+-------+-----------+--------------+-------------+
| UE23CS352A  |     7 |    117329 | 2026-09-17   | 2026-10-27  |
| UE23CS351A  |     6 |      1649 | 2026-09-16   | 2026-10-18  |
| UE23CS353A  |     5 |      1528 | 2026-09-26   | 2026-10-23  |
| UE23CS342A  |     4 |      1621 | 2026-09-21   | 2026-10-16  |
| UE23CS343A  |     4 |      1496 | 2026-09-23   | 2026-10-20  |
| UE23CS341A  |     4 |      1677 | 2026-09-20   | 2026-10-19  |
| UE23MA241B  |     3 |      1410 | 2026-09-25   | 2026-10-17  |
+-------------+-------+-----------+--------------+-------------+
(7 rows)
```

Q4 — Correlated subquery with NOT EXISTS

```
SELECT c.course_code, COUNT(*) AS students_without_group
FROM ENROLLMENT e
JOIN COURSE c ON c.course_id = e.course_id
WHERE NOT EXISTS (SELECT 1
                  FROM GROUP_MEMBERSHIP m JOIN STUDY_GROUP g ON g.group_id = m.group_id
                  WHERE m.student_id = e.student_id AND g.course_id = e.course_id)
GROUP BY c.course_code
ORDER BY students_without_group DESC;
```

```
+-------------+------------------------+
| course_code | students_without_group |
+-------------+------------------------+
| UE23MA241B  |                      7 |
| UE23CS344A  |                      6 |
| UE23CS343A  |                      5 |
| UE23CS352A  |                      5 |
| UE23CS342A  |                      4 |
| UE23CS353A  |                      4 |
| UE23CS341A  |                      3 |
| UE23CS351A  |                      1 |
+-------------+------------------------+
(8 rows)
```

Q5 — CTE + scalar subqueries

```
WITH uploads AS (
    SELECT s.student_id, s.full_name, COUNT(d.doc_id) AS notes
    FROM STUDENT s LEFT JOIN DOCUMENT d ON d.student_id = s.student_id
    WHERE s.role = 'student'
    GROUP BY s.student_id
)
SELECT full_name, notes, ROUND((SELECT AVG(notes) FROM uploads), 2) AS class_avg
FROM uploads
WHERE notes > (SELECT AVG(notes) FROM uploads)
ORDER BY notes DESC;
```

```
+-----------------+-------+-----------+
|    full_name    | notes | class_avg |
+-----------------+-------+-----------+
| Darshan P Pawar |     5 |      2.13 |
| Karthik Iyer    |     3 |      2.13 |
| Arjun Mehta     |     3 |      2.13 |
+-----------------+-------+-----------+
(3 rows)
```

Q6 — Window function RANK() OVER (PARTITION BY ...)

```
SELECT course_code, title, saves, course_rank
FROM (
    SELECT c.course_code, d.title, COUNT(b.student_id) AS saves,
           RANK() OVER (PARTITION BY c.course_code ORDER BY COUNT(b.student_id) DESC) AS course_rank
    FROM DOCUMENT d
    JOIN COURSE c ON c.course_id = d.course_id
    LEFT JOIN BOOKMARK b ON b.doc_id = d.doc_id
    GROUP BY c.course_code, d.doc_id
) ranked
WHERE course_rank = 1
ORDER BY saves DESC;
```

```
+-------------+-------------------------------------------------+-------+-------------+
| course_code |                      title                      | saves | course_rank |
+-------------+-------------------------------------------------+-------+-------------+
| UE23CS353A  | Transformers and Self-Attention Explained       |     6 |           1 |
| UE23CS343A  | Dynamic Programming Patterns                    |     5 |           1 |
| UE23CS351A  | Normalization from 1NF to BCNF                  |     4 |           1 |
| UE23CS352A  | Bias-Variance Trade-off and Regularization      |     4 |           1 |
| UE23CS341A  | TCP vs UDP and Congestion Control               |     3 |           1 |
| UE23CS342A  | CPU Scheduling Algorithms Compared              |     3 |           1 |
| UE23CS342A  | Virtual Memory, Paging and Page Replacement     |     3 |           1 |
| UE23MA241B  | Eigenvalues and Eigenvectors, Intuitively       |     3 |           1 |
| UE23CS344A  | Software Requirements: Functional vs Non-functional |     0 |           1 |
+-------------+-------------------------------------------------+-------+-------------+
(9 rows)
```

Q10 — Full-text search

```
SELECT d.title, ROUND(k.rank::NUMERIC, 3) AS rank, k.snippet
FROM fn_keyword_search(:darshan, 'deadlock OR "page fault"', 5) k
JOIN DOCUMENT d USING (doc_id);
```

```
+-------------------------------------------------------+-------+-------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------+
|                        title                          | rank  |
snippet
|
+-------------------------------------------------------+-------+-------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------+
| Concurrency Control: Two-Phase Locking and Deadlocks  | 0.667 | abort, which prevents cascading rollbacks because nobody
reads uncommitted data.\r
+|
|                                                       |       | \r
+|
|                                                       |       | 2PL has a price: ⟦deadlock⟧. Example: T1 holds X on
account A and requests B, while T2 holds … waiting for a lock held by T2. A cycle in the graph means ⟦deadlock⟧. The system
then aborts a victim, usually the cheapest transaction to undo, and rolls it back |
| Virtual Memory, Paging and Page Replacement           | 0.375 | valid bit; if the ⟦page⟧ is not in memory, the access
causes a ⟦page⟧ ⟦fault⟧. The OS finds the ⟦page⟧ on disk, picks a free frame, loads it, updates
|
| Semaphores, Mutexes and the Producer-Consumer Problem | 0.167 | then cannot get the mutex to remove an item, so neither
can ever proceed: ⟦deadlock⟧. The rule is to wait on the counting semaphore first, then take the mutex
|
+-------------------------------------------------------+-------+-------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------+
(3 rows)
```

Q14 — INSERT, UPDATE, DELETE, INSERT ... SELECT inside a transaction, then ROLLBACK

```
BEGIN;
INSERT INTO TAG (tag_name) VALUES ('  Write Ahead Logging ') RETURNING tag_id, tag_name;   -- trigger
normalises the name
UPDATE DOCUMENT SET visibility = 'public' WHERE file_path = 'seed/er-model-mapping.md' RETURNING title,
visibility;
INSERT INTO DOCUMENT_TAG (doc_id, tag_id)
    SELECT d.doc_id, t.tag_id FROM DOCUMENT d, TAG t
    WHERE d.file_path = 'seed/acid-isolation-levels.md' AND t.tag_name = 'write-ahead-logging'
    RETURNING doc_id, tag_id;
DELETE FROM BOOKMARK WHERE student_id = :darshan RETURNING doc_id;
SELECT COUNT(*) AS darshan_bookmarks_inside_tx FROM BOOKMARK WHERE student_id = :darshan;
ROLLBACK;
SELECT COUNT(*) AS darshan_bookmarks_after_rollback FROM BOOKMARK WHERE student_id = :darshan;
```

```
BEGIN
+--------+--------------------+
| tag_id |      tag_name      |
+--------+--------------------+
|     79 | write-ahead-logging |
+--------+--------------------+
(1 row)

INSERT 0 1
+--------------------------------------+------------+
|                title                 | visibility |
+--------------------------------------+------------+
| ER Modelling and Mapping to Relations | public     |
+--------------------------------------+------------+
(1 row)

UPDATE 1
+--------+--------+
| doc_id | tag_id |
+--------+--------+
|      5 |     79 |
+--------+--------+
(1 row)

INSERT 0 1
+--------+
| doc_id |
+--------+
|     10 |
|     23 |
|     18 |
|      5 |
|     11 |
|     20 |
+--------+
(6 rows)

DELETE 6
+----------------------------+
| darshan_bookmarks_inside_tx |
… (trimmed)
```

Q15 — Stored procedure + trigger rejecting a full group

```
-- CALL arguments cannot be sub-queries, so look the ids up first (\gset stores them in psql variables)
SELECT student_id AS me         FROM STUDENT     WHERE email = 'darshan.pawar@scholarmesh.test' \gset
SELECT student_id AS tanvi      FROM STUDENT     WHERE email = 'tanvi.hegde@scholarmesh.test' \gset
SELECT group_id   AS open_group FROM STUDY_GROUP WHERE group_name = 'Gradient Descenders' \gset
SELECT group_id   AS full_group FROM STUDY_GROUP WHERE group_name = 'Query Optimizers' \gset
BEGIN;
CALL sp_join_group(:me, :open_group);
SELECT group_name, member_count, seats_left FROM v_group_overview WHERE group_id = :open_group;
SAVEPOINT before_full;
CALL sp_join_group(:tanvi, :full_group);   -- expected: ERROR  This study group is full (4 of 4 seats taken)
ROLLBACK TO SAVEPOINT before_full;
ROLLBACK;
```

```
BEGIN
CALL
+--------------------+-------------+-----------+
|     group_name     | member_count | seats_left |
+--------------------+-------------+-----------+
| Gradient Descenders |           4 |         2 |
+--------------------+-------------+-----------+
(1 row)

SAVEPOINT
ERROR:  This study group is full (4 of 4 seats taken)
CONTEXT:  PL/pgSQL function fn_membership_check() line 13 at RAISE
SQL statement "INSERT INTO GROUP_MEMBERSHIP (group_id, student_id, role) VALUES (p_group, p_student, 'member')"
PL/pgSQL function sp_join_group(integer,integer) line 10 at SQL statement
ROLLBACK
ROLLBACK
```

Q16 — EXPLAIN ANALYZE — indexes in use

```
SET enable_seqscan = off;   -- the tables are tiny, so ask the planner to show the index path it would use at scale
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT group_id FROM GROUP_MEMBERSHIP WHERE student_id = 5;
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT doc_id FROM DOCUMENT WHERE search_tsv @@ websearch_to_tsquery('english', 'deadlock');
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF, SUMMARY OFF)
SELECT chunk_id FROM DOC_CHUNK
ORDER BY embedding <=> (SELECT embedding FROM DOC_CHUNK ORDER BY chunk_id LIMIT 1)
LIMIT 5;
RESET enable_seqscan;
```

```
--- B-tree on GROUP_MEMBERSHIP(student_id): "which groups is this student in?" (Phase 1 had no index here)
SET
+-------------------------------------------------------------------------+
|                              QUERY PLAN                                 |
+-------------------------------------------------------------------------+
| Bitmap Heap Scan on group_membership (actual rows=1 loops=1)            |
|   Recheck Cond: (student_id = 5)                                        |
|   Heap Blocks: exact=1                                                  |
|   ->  Bitmap Index Scan on idx_membership_student (actual rows=1 loops=1) |
|         Index Cond: (student_id = 5)                                    |
+-------------------------------------------------------------------------+
(5 rows)

--- GIN on DOCUMENT.search_tsv: full-text match
+-------------------------------------------------------------------------+
|                               QUERY PLAN                                |
+-------------------------------------------------------------------------+
| Bitmap Heap Scan on document (actual rows=3 loops=1)                    |
|   Recheck Cond: (search_tsv @@ '''deadlock'''::tsquery)                 |
|   Heap Blocks: exact=3                                                  |
|   ->  Bitmap Index Scan on idx_document_search_tsv (actual rows=3 loops=1) |
|         Index Cond: (search_tsv @@ '''deadlock'''::tsquery)             |
+-------------------------------------------------------------------------+
(5 rows)

--- HNSW on DOC_CHUNK.embedding: approximate nearest neighbours
+-------------------------------------------------------------------------------------------------+
|                                          QUERY PLAN                                             |
+-------------------------------------------------------------------------------------------------+
| Limit (actual rows=5 loops=1)                                                                   |
|   InitPlan 1                                                                                    |
|     ->  Limit (actual rows=1 loops=1)                                                           |
|           ->  Index Scan using doc_chunk_pkey on doc_chunk doc_chunk_1 (actual rows=1 loops=1) |
|   ->  Index Scan using idx_chunk_embedding_hnsw on doc_chunk (actual rows=5 loops=1)            |
|         Order By: (embedding <=> (InitPlan 1).col1)                                             |
+-------------------------------------------------------------------------------------------------+
(6 rows)

RESET
```

5.13 Advanced database features — summary

Feature (brief requires ≥ 4)	Where	Used by the app for
Transactions	`withTransaction()`, procedures, savepoints in tests	Upload, joins, admin changes; race-safe seats
Views	4 views	Catalogue, group overview, centroids, interest profiles
Materialized view	`mv_course_engagement`	Admin analytics
Stored functions / procedures	12 functions, 3 procedures	Access control, all search modes, recommendations, joining/leaving groups, tag merge
Triggers	14 triggers	Chunking, seat limits, enrollment rule, admin hand-over, audit
Indexing	34 indexes (B-tree, composite, partial, expression, GIN, HNSW)	Every listing and search
Full-text search	Generated weighted `tsvector` + GIN	Keyword and hybrid search, highlighted snippets
Vector similarity search	pgvector + HNSW, cosine	Semantic/hybrid search, similar notes, matching
Window functions, CTEs	RANK, ROW_NUMBER, SUM OVER, COUNT OVER	Rank fusion, pagination totals, leaderboards
Semi-structured data	JSONB in AUDIT_LOG	Before/after images of changes
Role-based privileges	GRANT/REVOKE, SECURITY DEFINER	Least-privilege app login

6. NoSQL / Vector database commands

The non-relational side of ScholarMesh is a **vector database**: pgvector adds a `vector` data type, distance operators and approximate-nearest-neighbour indexes to PostgreSQL. Semi-structured audit records use `JSONB` (schema-less documents inside a column). §6.2 lists the commands; §6.3 shows them running on the real data.

6.1 Vector data model

What is embedded	(a) every **passage** of every note — the text is split by trigger T4 using `fn_chunk_text` into chunks of at most 800 characters (paragraphs kept together, long paragraphs split on sentence boundaries); (b) every **search query**, stored in `SEARCH_LOG.query_embedding` for interest profiles.
Model	`sentence-transformers/all-MiniLM-L6-v2` (ONNX, 8-bit quantised), run locally in Node.js with Transformers.js — no API key and no note text leaves the server. Mean pooling + L2 normalisation → 384-dimensional unit vectors. Chosen after comparing three 384-d models on sample questions: it gave the clearest gap between relevant and unrelated notes.
Storage	`DOC_CHUNK.embedding vector(384)` (pgvector), `NULL` until the embedding worker fills it; `DOCUMENT.embedding_status` tracks `pending → ready` (or `failed`).
Index	`HNSW (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64)` — a navigable graph that finds approximate nearest neighbours without scanning every vector (EXPLAIN in §5.12, Q16).
Similarity metric	Cosine distance, operator `<=>`; the app reports *similarity = 1 − distance*.
Retrieval	`fn_semantic_search`: take the nearest passages among documents the student may see, keep the best passage per document, drop matches below 0.28 similarity, return the passage as the snippet.

6.2 pgvector and JSONB commands

Taken directly from the project files (file name and line numbers shown).

V1 — Enable the vector extension

Adds the `vector` type, the distance operators and the HNSW index method.

db/02_phase2_alter.sql

```
8 CREATE EXTENSION IF NOT EXISTS vector;
```

V2 — Store vectors next to the relational data

Every passage of a note gets a 384-number embedding. `doc_id` is a real foreign key, so deleting a note deletes its vectors in the same transaction.

db/02_phase2_alter.sql

```
90 CREATE TABLE DOC_CHUNK (
91     chunk_id     SERIAL PRIMARY KEY,
92     doc_id       INTEGER NOT NULL REFERENCES DOCUMENT(doc_id) ON DELETE CASCADE,
93     chunk_index  INTEGER NOT NULL CHECK (chunk_index >= 0),
94     chunk_text   TEXT    NOT NULL CHECK (length(chunk_text) > 0),
95     embedding    vector(384),            -- NULL until the embedding worker fills it
96     embedded_at  TIMESTAMPTZ,
97     CONSTRAINT uq_chunk_per_doc UNIQUE (doc_id, chunk_index),
98     CONSTRAINT chk_chunk_embedded CHECK ((embedding IS NULL) = (embedded_at IS NULL))
99 );
```

V3 — Keep the vector of every search

Used to build each student's interest profile.

db/02_phase2_alter.sql

```
77 ALTER TABLE SEARCH_LOG
78     ADD COLUMN search_mode     VARCHAR(10) NOT NULL DEFAULT 'hybrid',
79     ADD COLUMN result_count    INTEGER,
80     ADD COLUMN query_embedding vector(384),
81     ADD CONSTRAINT chk_search_mode  CHECK (search_mode IN ('keyword','semantic','hybrid')),
82     ADD CONSTRAINT chk_search_query CHECK (length(btrim(query_text)) >= 2),
83     ADD CONSTRAINT chk_search_count CHECK (result_count IS NULL OR result_count >= 0);
```

V4 — Index the vectors — HNSW with cosine distance

A navigable graph index: nearest neighbours are found without comparing the question with every stored vector.

db/02_phase2_alter.sql

```
121 CREATE INDEX idx_chunk_embedding_hnsw ON DOC_CHUNK
122     USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
```

V5 — Write embeddings with one set-based UPDATE

The API computes all of a note's vectors and stores them in one statement; `unnest` pairs each chunk id with its vector.

web/src/lib/repos/embedding-jobs.ts

```
16       // one set-based UPDATE for all chunks (unnest pairs ids with vectors)
17       await client.query(
18         `UPDATE DOC_CHUNK c
19          SET embedding = v.emb::vector, embedded_at = CURRENT_TIMESTAMP
20          FROM unnest($1::INT[], $2::TEXT[]) AS v(id, emb)
21          WHERE c.chunk_id = v.id`,
22         [chunks.map((c) => c.chunk_id), vectors.map(toVectorLiteral)],
23       );
```

V6 — Nearest-neighbour search (k-NN)

`ORDER BY embedding <=> query` uses the HNSW index; the best passage per note is kept and weak matches (similarity below 0.28) are dropped.

db/04_functions.sql

```
124 CREATE OR REPLACE FUNCTION fn_semantic_search(p_student INT, p_query_vec vector, p_limit INT DEFAULT 10,
125                                              p_min_similarity REAL DEFAULT 0.28,
126                                              p_course INT DEFAULT NULL, p_tag TEXT DEFAULT NULL)
127 RETURNS TABLE (doc_id INT, similarity REAL, snippet TEXT, chunk_index INT)
128 LANGUAGE sql STABLE AS $$
129     WITH nearest_chunks AS (
130         SELECT c.doc_id, c.chunk_index, c.chunk_text, (c.embedding <=> p_query_vec) AS distance
131         FROM DOC_CHUNK c
132         JOIN fn_search_scope(p_student, p_course, p_tag) v ON v.doc_id = c.doc_id
133         WHERE c.embedding IS NOT NULL
134         ORDER BY c.embedding <=> p_query_vec
135         LIMIT p_limit * 8
136     ), best_per_doc AS (
137         SELECT DISTINCT ON (n.doc_id) n.doc_id, n.chunk_index, n.chunk_text, n.distance
138         FROM nearest_chunks n
139         ORDER BY n.doc_id, n.distance
140     )
141     SELECT b.doc_id, (1 - b.distance)::REAL, b.chunk_text, b.chunk_index
142     FROM best_per_doc b
143     WHERE 1 - b.distance >= p_min_similarity
144     ORDER BY b.distance
145     LIMIT p_limit
146 $$;
```

V7 — Aggregate vectors — one centroid per note

pgvector's `AVG` turns a note's passage vectors into one vector, used for “more like this” and interest profiles.

db/03_views.sql

```
69 CREATE VIEW v_doc_centroid AS
70 SELECT doc_id, AVG(embedding) AS centroid, COUNT(*)::INT AS chunk_count
71 FROM DOC_CHUNK
72 WHERE embedding IS NOT NULL
73 GROUP BY doc_id;
```

V8 — Semi-structured JSONB documents

The audit log keeps a schema-less copy of each changed row, before and after.

db/02_phase2_alter.sql

```
104 CREATE TABLE AUDIT_LOG (
105     audit_id    BIGSERIAL PRIMARY KEY,
106     table_name  VARCHAR(40) NOT NULL,
107     action      VARCHAR(10) NOT NULL CHECK (action IN ('INSERT','UPDATE','DELETE')),
108     row_pk      VARCHAR(40) NOT NULL,
109     actor_id    INTEGER REFERENCES STUDENT(student_id) ON DELETE SET NULL,
110     old_data    JSONB,
111     new_data    JSONB,
112     logged_at   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
113 );
```

V9 — Build and read JSONB inside a trigger

`to_jsonb` turns a row into a document, `-` removes secret keys, `->>` reads one field.

db/05_triggers.sql

```
259     IF TG_OP <> 'INSERT' THEN
260         v_old := to_jsonb(OLD) - 'password_hash' - 'content' - 'search_tsv';
261     END IF;
262     IF TG_OP <> 'DELETE' THEN
263         v_new := to_jsonb(NEW) - 'password_hash' - 'content' - 'search_tsv';
264     END IF;
265     v_pk := COALESCE(v_new, v_old) ->> TG_ARGV[0];
```

6.3 Vector queries on the real data — with output

From `db/10_important_queries.sql`, executed against the seeded database; the output below each query is the real psql output (long results trimmed). Data changes run inside `BEGIN … ROLLBACK`.

Q11 — Vector similarity search

```
SELECT d.title, ROUND(s.similarity::NUMERIC, 3) AS similarity, LEFT(s.snippet, 70) AS best_passage
FROM fn_semantic_search(:darshan,
        (SELECT query_embedding FROM SEARCH_LOG WHERE query_text = 'how do models focus on the important words'
LIMIT 1), 5) s
JOIN DOCUMENT d USING (doc_id);
```

```
+-----------------------------------------+------------+-------------------------------------------------------------------
------+
|                  title                  | similarity |                           best_passage
|
+-----------------------------------------+------------+-------------------------------------------------------------------
------+
| Word Embeddings: word2vec and GloVe     |      0.453 | The idea comes from the distributional hypothesis: you shall know
a wo |
| Transformers and Self-Attention Explained |      0.435 | Take "the animal did not cross the street because it was tired".
The q |
| ER Modelling and Mapping to Relations   |      0.335 | An ER (entity-relationship) model is a diagram of the data drawn
befor |
+-----------------------------------------+------------+-------------------------------------------------------------------
------+
(3 rows)
```

Q12 — Hybrid search (Reciprocal Rank Fusion)

```
SELECT d.title, h.keyword_rank, h.semantic_rank, ROUND(h.score::NUMERIC, 4) AS rrf_score
FROM fn_hybrid_search(:darshan, 'gradient',
        (SELECT query_embedding FROM SEARCH_LOG WHERE query_text = 'why do gradients vanish in deep networks'
LIMIT 1), 6) h
JOIN DOCUMENT d USING (doc_id);
```

```
+-----------------------------------------+--------------+---------------+-----------+
|                  title                  | keyword_rank | semantic_rank | rrf_score |
+-----------------------------------------+--------------+---------------+-----------+
| RNNs, LSTMs and the Vanishing Gradient  |            1 |             1 |    0.0328 |
| Linear Regression and Gradient Descent  |            2 |             3 |    0.0320 |
| Backpropagation Without the Mystery     |            3 |             2 |    0.0320 |
| Transformers and Self-Attention Explained |            4 |             4 |    0.0313 |
| Convolutional Neural Networks Basics    |              |             5 |    0.0154 |
+-----------------------------------------+--------------+---------------+-----------+
(5 rows)
```

Q13 — Study-group and partner matching

```
SELECT group_name, course_code, seats_left, ROUND(similarity::NUMERIC, 3) AS similarity, shared_tags,
ROUND(match_score::NUMERIC, 3) AS match_score
FROM fn_recommend_groups(:darshan, 5);
SELECT full_name, shared_courses, ROUND(similarity::NUMERIC, 3) AS similarity FROM
fn_recommend_partners(:darshan, 5);
```

```
+-------------------------+-------------+------------+------------+-------------------------------------------------------------
-----------------------------------------+-------------+
|        group_name       | course_code | seats_left | similarity |                                      shared_tags
| match_score |
+-------------------------+-------------+------------+------------+-------------------------------------------------------------
-----------------------------------------+-------------+
| Attention Is All We Need | UE23CS353A  |          4 |      0.695 | {attention,backpropagation,gradient-descent,neural-
networks,nlp,transformers,word-embeddings} |       0.696 |
| Gradient Descenders      | UE23CS352A  |          3 |      0.661 | {classification,decision-trees,ensemble-learning,exam-
prep,overfitting,regularization}        |       0.613 |
| Kernel Trick Club        | UE23CS352A  |          2 |      0.643 | {classification,decision-trees,ensemble-learning}
|       0.540 |
+-------------------------+-------------+------------+------------+-------------------------------------------------------------
-----------------------------------------+-------------+
(3 rows)

+---------------+-------------------------------------------------------------+------------+
|   full_name   |                       shared_courses                        | similarity |
+---------------+-------------------------------------------------------------+------------+
| Siddharth Rao | {UE23CS343A,UE23CS352A,UE23CS353A}                          |      0.644 |
| Meera Pillai  | {UE23CS343A,UE23CS351A,UE23CS352A,UE23CS353A,UE23MA241B} |      0.634 |
| Diya Shah     | {UE23CS343A,UE23CS351A,UE23CS352A}                          |      0.613 |
| Ishita Das    | {UE23CS351A,UE23CS352A,UE23CS353A}                          |      0.599 |
| Sneha Kulkarni | {UE23CS352A,UE23CS353A,UE23MA241B}                          |      0.584 |
+---------------+-------------------------------------------------------------+------------+
(5 rows)
```

6.4 Choosing the similarity cut-off

The threshold was measured, not guessed. Off-topic questions never scored above 0.27 while relevant notes scored 0.38+:

Question	Best similarity	Best note	Returned at 0.28?
what happens when i type a url	0.52	What Happens During a DNS Lookup	yes
how do models decide which words matter	0.51	Word Embeddings: word2vec and GloVe	yes
how to stop overfitting	0.45	Bias-Variance Trade-off and Regularization	yes
why does my table have duplicate data	0.38	Transactions, ACID … / Normalization	yes
quantum computing basics	0.27	(unrelated)	no → logged as a content gap
agile sprint planning	0.23	(unrelated)	no
photosynthesis in plants	0.19	(unrelated)	no

6.5 Hybrid search — Reciprocal Rank Fusion

Keyword search is precise for exact terms ("BCNF"); semantic search understands paraphrases. `fn_hybrid_search` runs both and merges them with **Reciprocal Rank Fusion**: `score = 1/(60 + keyword_rank) + 1/(60 + semantic_rank)` (a missing rank contributes 0). A note found by both methods rises to the top; a note found by only one still appears. Example: "*bcnf decomposition*" returns nothing in keyword mode (no note contains "decomposition") but the normalization note in semantic and hybrid mode (screenshots in §8.2).

6.6 Interest profiles and study-group matching

`v_doc_centroid` averages each document's passage vectors. `v_student_interest` averages, per student, the centroids of notes they uploaded or bookmarked and the vectors of their searches from the last 120 days. A group's interest is the average of its members' interest vectors. `fn_recommend_groups` considers only groups in the student's courses that still have a seat and ranks them by

$$\text{match} = 0.7 \times \text{cosine}(\text{student interest}, \text{group interest}) + 0.3 \times \text{Jaccard}(\text{student topics}, \text{group topics})$$

where "topics" are the tags of notes each person uploaded or bookmarked *within that group's course*. `fn_recommend_partners` ranks classmates (students sharing a course) by interest similarity.

6.7 Embedding pipeline

1. Upload: one transaction inserts the `DOCUMENT`; trigger T4 writes its `DOC_CHUNK` rows (no vectors yet); tags are linked.
2. After commit the API embeds the new chunks in one batch and writes them with a single set-based `UPDATE … FROM unnest(…)`; the document becomes `ready`.
3. If the model is unavailable the note is still saved and keyword-searchable (`failed`); an admin re-runs the job later. Editing the text re-chunks it and resets it to `pending`.

7. Next.js — Project tree structure

The application lives in `web/` and uses the Next.js **App Router**: every folder under `src/app` is a URL, a `page.tsx` file is a page and a `route.ts` file is an API endpoint. Folders in brackets such as `(app)` group pages under a shared layout without becoming part of the URL; `[id]` is a URL parameter. The tree below is generated from the repository (`git ls-files`); the comment beside each page or route is the URL it serves.

7.1 Project tree

```
DBMS-Mini-Project/
├── db/                                   # SQL, run in order by npm run db:setup (§5)
│   ├── 01_schema.sql
│   ├── 02_phase2_alter.sql
│   ├── 03_views.sql
│   ├── 04_functions.sql
│   ├── 05_triggers.sql
│   ├── 06_security.sql
│   ├── 07_seed_core.sql
│   ├── 08a_seed_notes.sql
│   ├── 08b_seed_notes.sql
│   ├── 08c_seed_notes.sql
│   ├── 08d_seed_notes.sql
│   ├── 09_seed_activity.sql
│   └── 10_important_queries.sql
├── docs/                                 # report, guide, diagrams, screenshots, demo video
├── web/                                  # the Next.js application
│   ├── public/                           # static files
│   ├── scripts/                          # database setup, embedding job, screenshots, demo video, PDF rendering
│   │   ├── db-setup.ts
│   │   ├── demo-video.ts
│   │   ├── embed-pending.ts
│   │   ├── render-html.ts
│   │   └── screenshots.ts
│   ├── src/                              # application source
│   │   ├── app/                          # App Router: folders are URLs
│   │   │   ├── (app)/                    # pages for signed-in users (shared layout + navigation)
│   │   │   │   ├── admin/                # admin role only
│   │   │   │   │   ├── audit/
│   │   │   │   │   │   └── page.tsx      # /admin/audit
│   │   │   │   │   ├── content/
│   │   │   │   │   │   ├── admin-forms.tsx
│   │   │   │   │   │   └── page.tsx      # /admin/content
│   │   │   │   │   ├── users/
│   │   │   │   │   │   └── page.tsx      # /admin/users
│   │   │   │   │   └── page.tsx          # /admin
│   │   │   │   ├── courses/
│   │   │   │   │   ├── enroll-button.tsx
│   │   │   │   │   └── page.tsx          # /courses
│   │   │   │   ├── dashboard/
│   │   │   │   │   └── page.tsx          # /dashboard
│   │   │   │   ├── documents/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── page.tsx      # /documents/:id
│   │   │   │   │   ├── new/
│   │   │   │   │   │   └── page.tsx      # /documents/new
│   │   │   │   │   ├── document-form.tsx
│   │   │   │   │   └── page.tsx          # /documents
│   │   │   │   ├── groups/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── page.tsx      # /groups/:id
│   │   │   │   │   ├── group-form.tsx
│   │   │   │   │   └── page.tsx          # /groups
│   │   │   │   ├── search/
│   │   │   │   │   ├── page.tsx          # /search
│   │   │   │   │   └── search-client.tsx
│   │   │   │   └── layout.tsx
│   │   │   ├── (auth)/                   # login and registration pages
│   │   │   │   ├── login/
│   │   │   │   │   └── page.tsx          # /login
│   │   │   │   ├── register/
```

```
│   │   │   │   │   └── page.tsx                  # /register
│   │   │   │   ├── auth-forms.tsx
│   │   │   │   └── layout.tsx
│   │   │   ├── api/                              # REST API (route handlers): the only path to the database
│   │   │   │   ├── admin/
│   │   │   │   │   ├── courses/
│   │   │   │   │   │   ├── [id]/
│   │   │   │   │   │   │   └── route.ts          # /api/admin/courses/:id
│   │   │   │   │   │   └── route.ts              # /api/admin/courses
│   │   │   │   │   ├── embeddings/
│   │   │   │   │   │   └── route.ts              # /api/admin/embeddings
│   │   │   │   │   ├── refresh-stats/
│   │   │   │   │   │   └── route.ts              # /api/admin/refresh-stats
│   │   │   │   │   ├── stats/
│   │   │   │   │   │   └── route.ts              # /api/admin/stats
│   │   │   │   │   ├── tags/
│   │   │   │   │   │   └── merge/
│   │   │   │   │   │       └── route.ts          # /api/admin/tags/merge
│   │   │   │   │   └── users/
│   │   │   │   │       └── [id]/
│   │   │   │   │           └── route.ts          # /api/admin/users/:id
│   │   │   │   ├── auth/
│   │   │   │   │   ├── login/
│   │   │   │   │   │   └── route.ts              # /api/auth/login
│   │   │   │   │   ├── logout/
│   │   │   │   │   │   └── route.ts              # /api/auth/logout
│   │   │   │   │   └── register/
│   │   │   │   │       └── route.ts              # /api/auth/register
│   │   │   │   ├── courses/
│   │   │   │   │   └── route.ts                  # /api/courses
│   │   │   │   ├── documents/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   ├── bookmark/
│   │   │   │   │   │   │   └── route.ts          # /api/documents/:id/bookmark
│   │   │   │   │   │   ├── file/
│   │   │   │   │   │   │   └── route.ts          # /api/documents/:id/file
│   │   │   │   │   │   └── route.ts              # /api/documents/:id
│   │   │   │   │   └── route.ts                  # /api/documents
│   │   │   │   ├── enrollments/
│   │   │   │   │   └── route.ts                  # /api/enrollments
│   │   │   │   ├── groups/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   ├── members/
│   │   │   │   │   │   │   └── [studentId]/
│   │   │   │   │   │   │       └── route.ts      # /api/groups/:id/members/:studentId
│   │   │   │   │   │   ├── membership/
│   │   │   │   │   │   │   └── route.ts          # /api/groups/:id/membership
│   │   │   │   │   │   └── route.ts              # /api/groups/:id
│   │   │   │   │   └── route.ts                  # /api/groups
│   │   │   │   ├── recommendations/
│   │   │   │   │   └── route.ts                  # /api/recommendations
│   │   │   │   └── search/
│   │   │   │       └── route.ts                  # /api/search
│   │   │   ├── error.tsx
│   │   │   ├── favicon.ico
│   │   │   ├── globals.css
│   │   │   ├── layout.tsx                        # root layout: fonts, metadata
│   │   │   ├── not-found.tsx
│   │   │   └── page.tsx                          # landing page  /
│   │   ├── components/                           # shared React components
│   │   │   ├── action-button.tsx
│   │   │   ├── api-client.ts
│   │   │   ├── app-nav.tsx
│   │   │   ├── brand.tsx
│   │   │   ├── charts.tsx
│   │   │   ├── document-card.tsx
│   │   │   └── ui.tsx
│   │   ├── lib/                                  # server-side logic
│   │   │   ├── auth/                             # bcrypt passwords, JWT sessions, current user
│   │   │   │   ├── current-user.ts
│   │   │   │   ├── password.ts
│   │   │   │   └── token.ts
│   │   │   ├── repos/                            # all SQL used by the app (parameterised)
│   │   │   │   ├── courses.ts
│   │   │   │   ├── documents.ts
```

```
│   │   │   │   ├── embedding-jobs.ts
│   │   │   │   ├── groups.ts
│   │   │   │   ├── search.ts
│   │   │   │   ├── stats.ts
│   │   │   │   └── students.ts
│   │   │   ├── chart-palette.ts
│   │   │   ├── db.ts                          # connection pool, query(), withTransaction()
│   │   │   ├── embeddings.ts                  # local embedding model (384-d)
│   │   │   ├── env.ts
│   │   │   ├── errors.ts
│   │   │   ├── files.ts                       # upload storage, PDF/text extraction
│   │   │   ├── format.ts
│   │   │   ├── http.ts                        # JSON responses, error mapping, CSRF check
│   │   │   ├── rate-limit.ts
│   │   │   ├── safe-redirect.ts
│   │   │   ├── types.ts
│   │   │   └── validation.ts                  # zod schemas for every request
│   │   └── proxy.ts                           # first gate: signed session cookie
│   ├── tests/                                 # Vitest + Playwright
│   │   ├── db/                                # database integration tests (rolled back)
│   │   │   ├── constraints.test.ts
│   │   │   ├── functions.test.ts
│   │   │   ├── helpers.ts
│   │   │   ├── security-concurrency.test.ts
│   │   │   └── triggers.test.ts
│   │   ├── e2e/                               # end-to-end browser tests
│   │   │   └── app.spec.ts
│   │   ├── unit/                              # unit tests
│   │   │   ├── auth.test.ts
│   │   │   ├── http.test.ts
│   │   │   └── validation.test.ts
│   │   └── setup-env.ts
│   ├── .env.example                           # DATABASE_URL + AUTH_SECRET template
│   ├── .gitignore
│   ├── AGENTS.md
│   ├── eslint.config.mjs
│   ├── next.config.ts                         # Next.js settings, security headers
│   ├── package-lock.json
│   ├── package.json
│   ├── playwright.config.ts
│   ├── README.md
│   ├── tsconfig.json
│   └── vitest.config.mts
├── .env.example                               # database passwords template (real .env is git-ignored)
├── docker-compose.yml                         # PostgreSQL 17 + pgvector container
└── README.md                                  # setup, demo logins, tests
```

7.2 How the parts connect — architecture

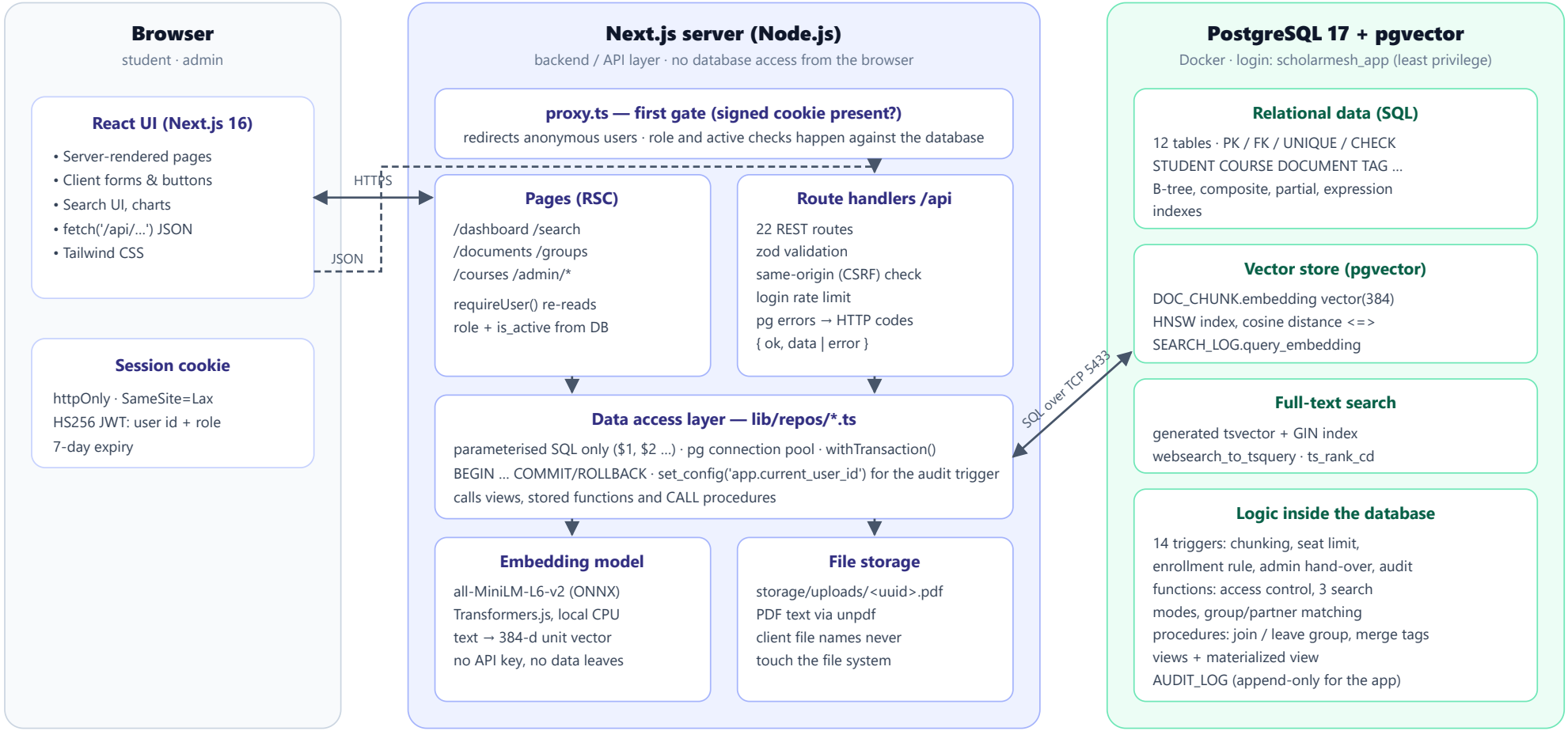


Figure 2 — Architecture. The browser never talks to the database; every read and write goes through server code that re-checks the user and uses parameterised SQL.

7.3 Request flow (example: a semantic search)

1. The browser calls `GET /api/search?q=…&mode=semantic` with the session cookie.
2. `proxy.ts` only checks that a validly signed cookie exists; the route handler re-reads the user from STUDENT (role, is_active) and validates the query with zod — so deactivation, deletion and promotion take effect immediately.
3. The server embeds the question with the local model (cached for repeated questions).
4. One SQL statement calls `fn_semantic_search`, which applies `fn_visible_doc_ids` and the HNSW index, and joins `v_document_catalog` for display fields.
5. The query and its vector are written to SEARCH_LOG (analytics and interest profile); results return as `{ ok, data }`.

7.4 API endpoints (34 handlers in 22 routes)

Endpoint	Methods	Who	Purpose
`/api/auth/register`, `/login`, `/logout`	POST	public / any	Account and session (rate-limited)
`/api/search`	GET	signed in	Keyword / semantic / hybrid search with filters
`/api/documents`	GET, POST	signed in	Library listing; multipart upload
`/api/documents/:id`	GET, PATCH, DELETE	viewer / owner / admin	Read, edit metadata, delete
`/api/documents/:id/file`, `/bookmark`	GET · POST, DELETE	viewer	Download; save / unsave
`/api/courses`, `/api/enrollments`	GET · POST, DELETE	student	Catalogue; enroll / drop
`/api/groups`, `/api/groups/:id`	GET, POST · GET, PATCH, DELETE	student / group admin	List, create, view, edit, delete groups
`/api/groups/:id/membership`	POST, DELETE	student	Join (procedure) / leave
`/api/groups/:id/members/:studentId`	PATCH, DELETE	group admin	Promote / demote / remove members
`/api/recommendations`	GET	student	Matched groups and partners
`/api/admin/stats`, `/refresh-stats`, `/embeddings`	GET · POST · POST	admin	Analytics, MV refresh, embedding job
`/api/admin/users/:id`	PATCH, DELETE	admin	Role, activation, deletion (not self)
`/api/admin/courses`, `/courses/:id`, `/tags/merge`	POST · PATCH, DELETE · POST	admin	Course catalogue; tag merge

7.5 Application security

- bcrypt password hashes; constant-cost login (a dummy hash is checked for unknown e-mails); 10 attempts per 15 minutes per account, plus per-IP limits when a trusted reverse proxy supplies the address (a client-sent `X-Forwarded-For` is never trusted).

- Session: HS256 JWT in an `httpOnly`, `SameSite=Lax` cookie (`Secure` in production); role and active flag re-read from the database on every request.
- CSRF: every POST/PATCH/DELETE must come from the same origin. XSS: React escapes output; search highlights use markers that are turned into `<mark>` elements, never injected as HTML.
- Uploads: body size checked before parsing (5 MB), extension whitelist, PDF magic-byte check, at most 150 pages and 300 000 characters, 30 uploads per hour per user, random server-side file names (no path traversal).
- Private notes never leak through aggregates: tag lists, trending topics, course note counts, interest profiles and group matching all count only notes the viewer may see. Group pages are visible only to members, students of the course and admins.
- Security headers (X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy); generic 500 messages, details only in server logs.
- Configuration only through environment variables, validated at start-up (`DATABASE_URL`, `AUTH_SECRET` ≥ 32 chars).

8. Sample screenshots — application code and demo

8.1 Application code

Excerpts of the real source files (file name and line numbers as in the repository), following one request from the browser to the database: the gate, the API route, the repository that runs the SQL, the transaction helper, the upload transaction and the React component that calls the API.

web/src/proxy.ts

```ts
const PUBLIC_API = new Set(["/api/auth/login", "/api/auth/register"]);

export async function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const session = await verifySession(request.cookies.get(SESSION_COOKIE)?.value, process.env.AUTH_SECRET ??
"");

  if (pathname.startsWith("/api/")) {
    if (!session && !PUBLIC_API.has(pathname)) {
      return NextResponse.json({ ok: false, error: { message: "Please log in first" } }, { status: 401 });
    }
    return NextResponse.next();
  }

  if (!session) {
    const login = new URL("/login", request.url);
    login.searchParams.set("next", pathname);
    return NextResponse.redirect(login);
  }
  return NextResponse.next();
}
```

Figure 3 — proxy.ts — the first gate: a page or API request without a valid signed session cookie is turned away before any page code runs.

web/src/app/api/search/route.ts

```ts
// GET /api/search?q=...&mode=hybrid|semantic|keyword&course=ID&tag=name
export const GET = route(async (request) => {
  const user = await requireApiUser();
  const params = Object.fromEntries(new URL(request.url).searchParams);
  const input = searchSchema.parse(params);
  return ok(await runSearch(user.student_id, input));
});
```

Figure 4 — Route handler for GET /api/search — the API layer: it re-reads the signed-in user, validates the request with zod and calls the search repository.

web/src/lib/repos/search.ts

```ts
export async function runSearch(studentId: number, req: SearchRequest): Promise<SearchResponse> {
  const started = performance.now();
  const vector = await safeEmbed(req.q);
  // without a model, fall back to keyword search rather than failing
  const mode: SearchMode = vector ? req.mode : "keyword";
  const vectorLiteral = vector ? toVectorLiteral(vector) : null;

  const { sql, params } = buildQuery(mode, studentId, req, vectorLiteral);
  const hits = await query<SearchHit>(sql, params);

  // every search is logged; the query vector feeds the student's interest profile
  await query(
    `INSERT INTO SEARCH_LOG (query_text, student_id, search_mode, result_count, query_embedding)
     VALUES ($1, $2, $3, $4, $5::vector)`,
    [req.q, studentId, mode, hits.length, vectorLiteral],
  );

  return { mode, hits, tookMs: Math.round(performance.now() - started) };
}
```

Figure 5 — Search repository — embeds the question, runs one stored search function in PostgreSQL and logs the search together with its vector.

web/src/lib/db.ts

```ts
export async function withTransaction<T>(
  work: (client: PoolClient) => Promise<T>,
  options: { actorId?: number | null } = {},
): Promise<T> {
  const client = await pool().connect();
  try {
    await client.query("BEGIN");
    if (options.actorId) {
      await client.query("SELECT set_config('app.current_user_id', $1, true)", [String(options.actorId)]);
    }
    const result = await work(client);
    await client.query("COMMIT");
    return result;
  } catch (error) {
    await client.query("ROLLBACK").catch(() => undefined);
    throw error;
  } finally {
    client.release();
  }
}
```

Figure 6 — Transactions — BEGIN … COMMIT on one pooled connection, ROLLBACK on any error; the acting user is handed to the audit trigger.

web/src/lib/repos/documents.ts

```ts
export async function createDocument(ownerId: number, doc: NewDocument): Promise<number> {
  return withTransaction(
    async (client) => {
      const { rows } = await client.query<{ doc_id: number }>(
        `INSERT INTO DOCUMENT (title, description, file_path, file_type, file_size_bytes, visibility, content, student_id, course_id)
         VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
         RETURNING doc_id`,
        [doc.title, doc.description, doc.file_path, doc.file_type, doc.file_size_bytes, doc.visibility, doc.content, ownerId, doc.course_id],
      );
      const docId = rows[0].doc_id;
      await replaceTags(client, docId, doc.tags);
      return docId;
    },
    { actorId: ownerId },
  );
}
```

Figure 7 — Upload — the note is inserted and its tags linked in one transaction; trigger T4 splits the text into passages.

web/src/app/(app)/search/search-client.tsx

```tsx
async function run(next: Query) {
  if (next.q.trim().length < 2) {
    setError("Type at least 2 characters.");
    return;
  }
  setQuery(next);
  setIsBusy(true);
  setError(null);
  const params = toParams(next);
  const requestId = ++latestRequest.current;
  const result = await apiFetch<SearchResponse>(`/api/search?${params}`);
  if (requestId !== latestRequest.current) return; // a newer search was started meanwhile
  setIsBusy(false);
  if (!result.ok) {
    setError(result.error.message);
    return;
  }
  setResponse(result.data);
  // keep the URL shareable without a server round trip
  window.history.replaceState(null, "", `/search?${params}`);
}
```

Figure 8 — React client component (search page) — calls the API, ignores out-of-date responses and keeps the URL shareable.

8.2 Application demo

Captured automatically from the production build by `web/scripts/screenshots.ts` (Playwright, 1440 px wide). A narrated **demo video** of the same flows (4 min 47 s, 1080p, with chapters) is in the repository at `docs/demo/ScholarMesh_Demo.mp4`, with its script in `docs/demo/demo-script.md`.

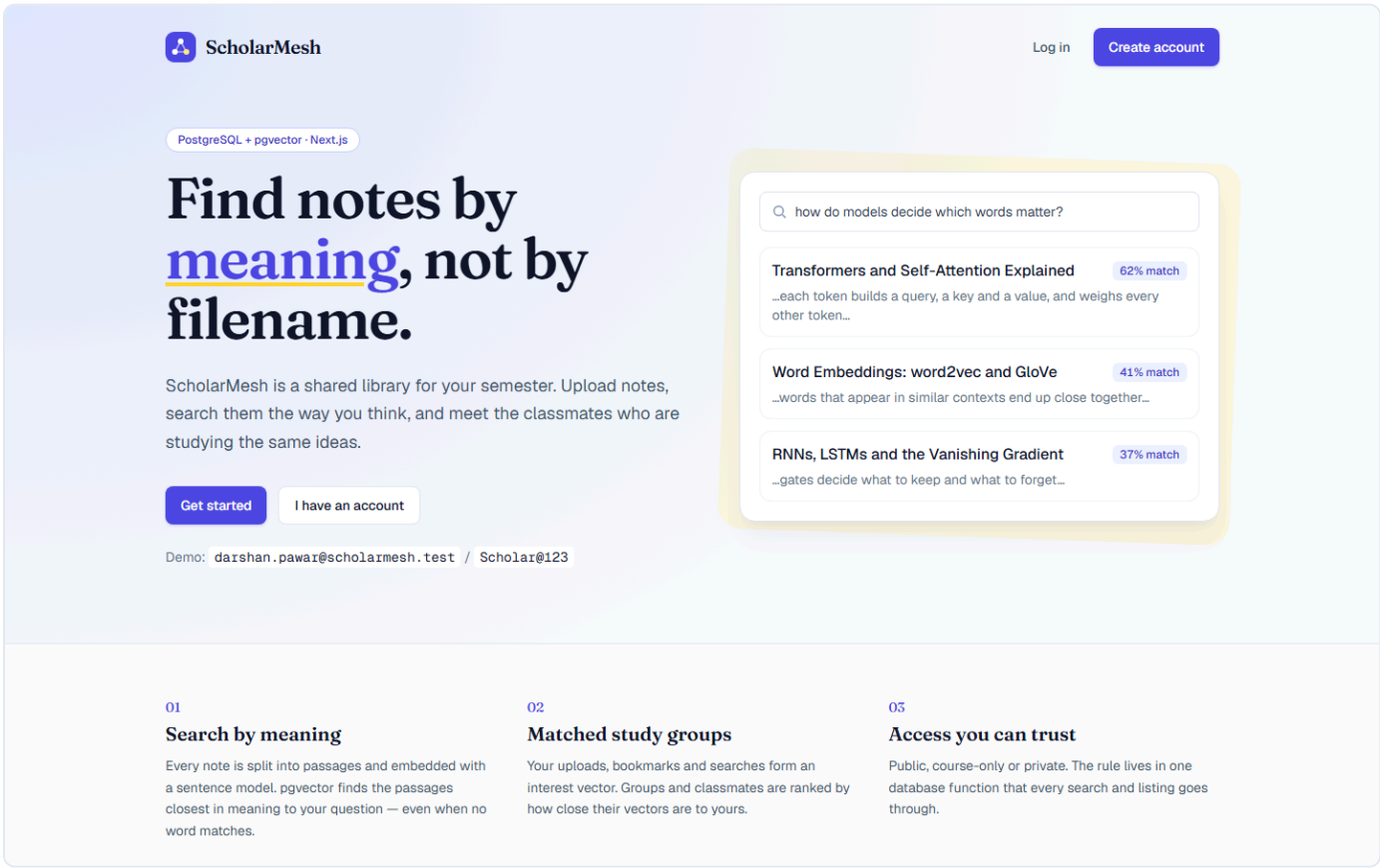


Figure 9 — Landing page (signed out)

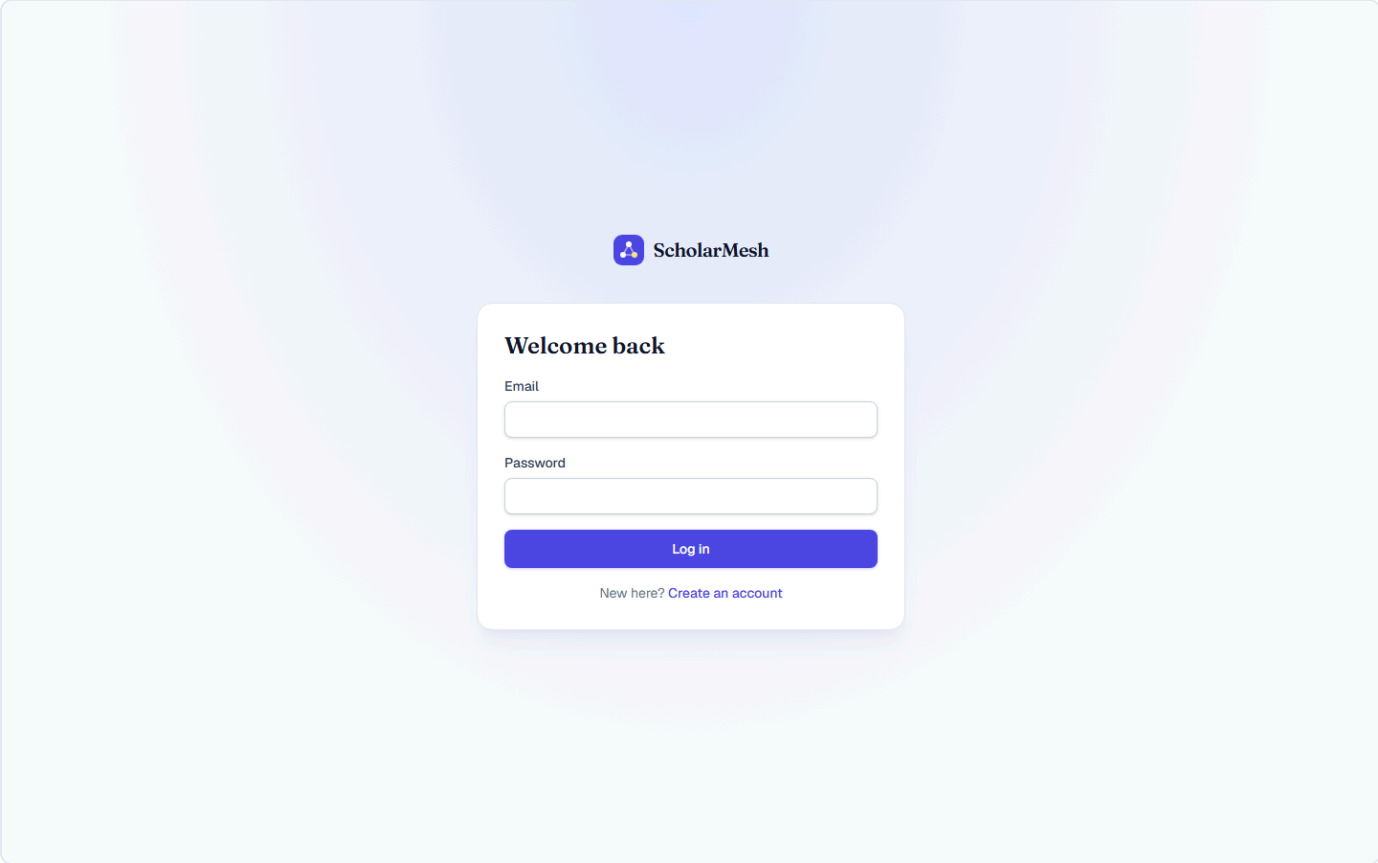


Figure 10 — Login with validation and error messages

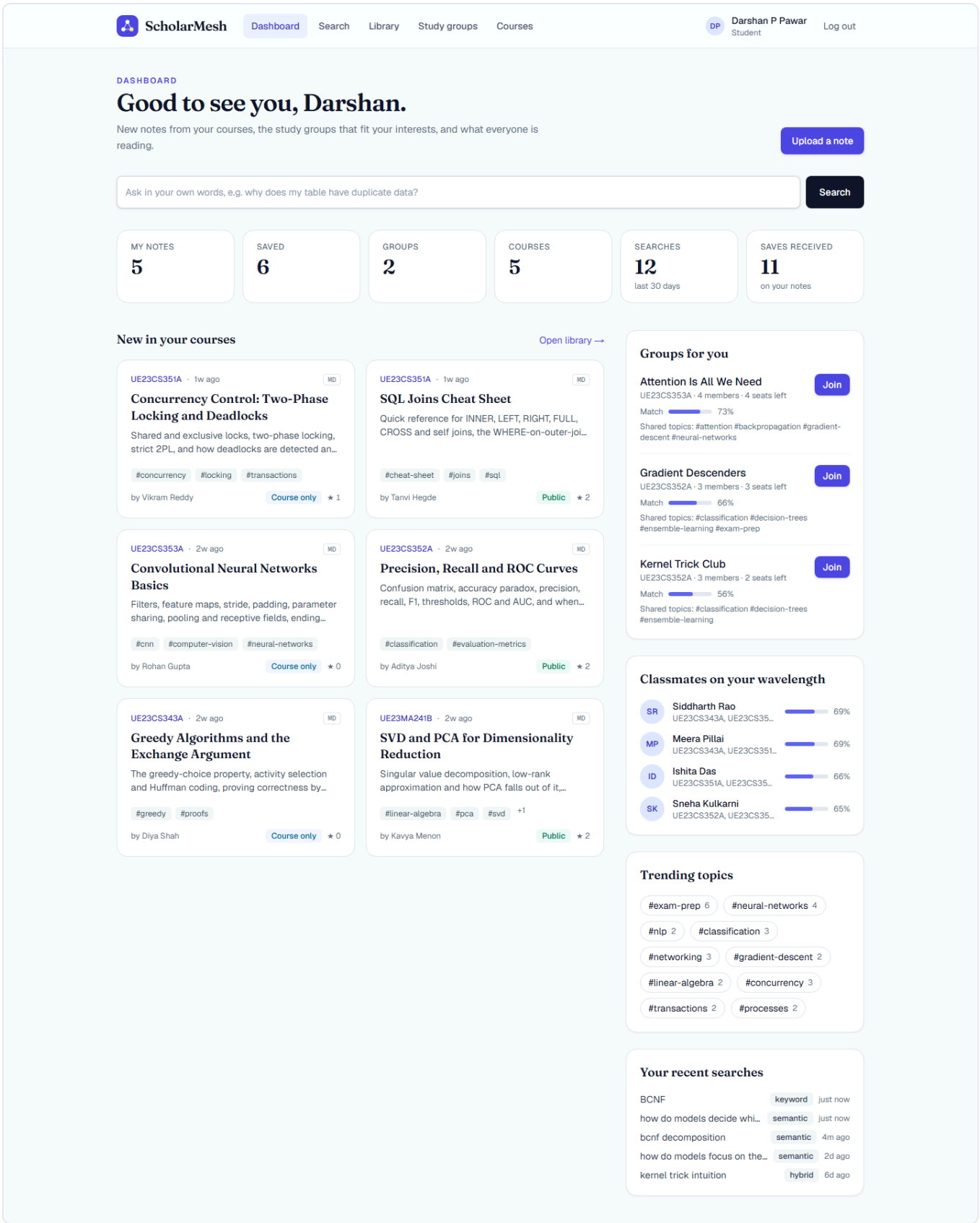


Figure 11 — Student dashboard — statistics (fn_student_stats), new notes in my courses, matched groups, classmates by interest similarity, trending topics (GROUP BY … HAVING), recent searches

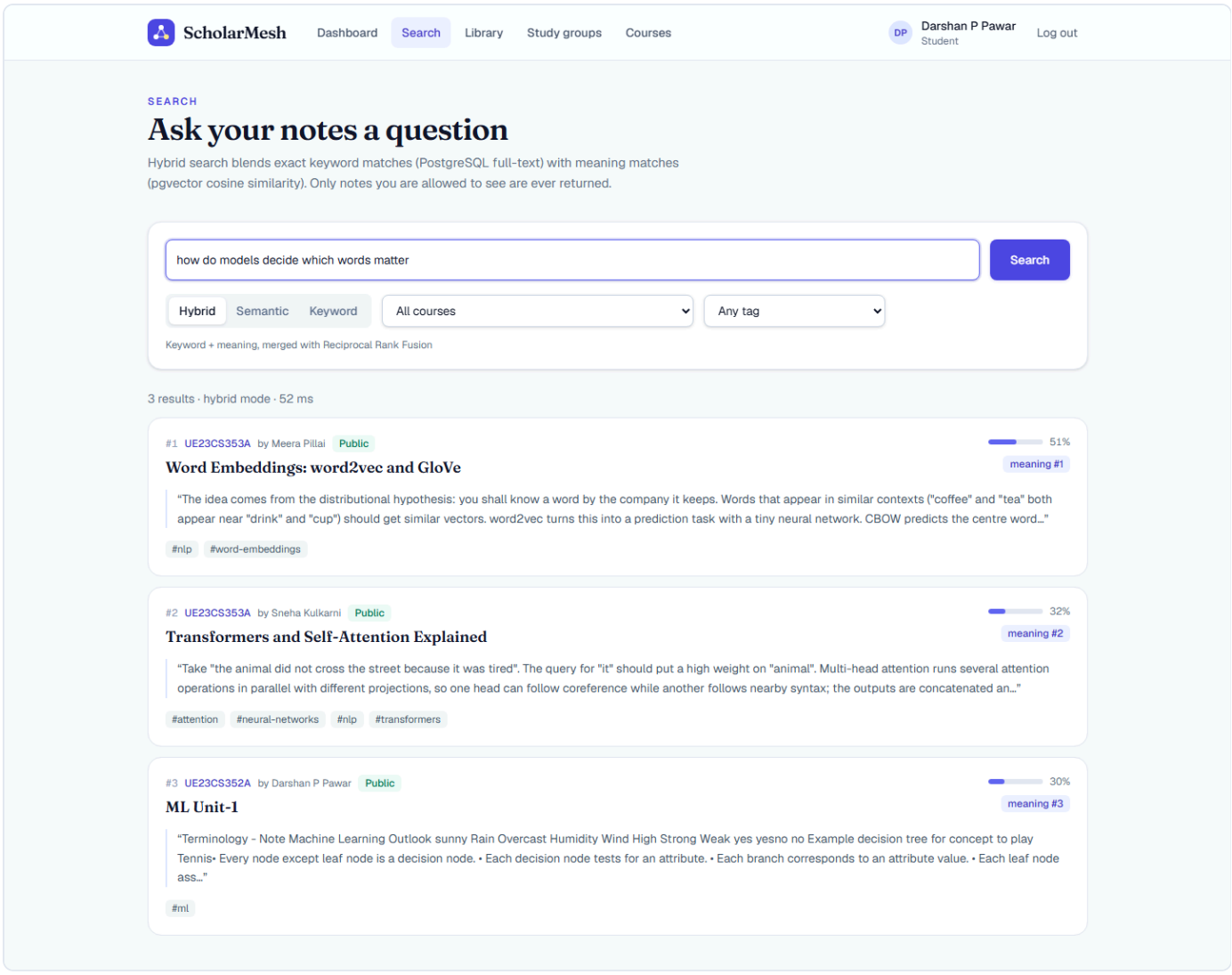


Figure 12 — Hybrid search — a question with no shared keywords; ranks from the semantic side and cosine similarity

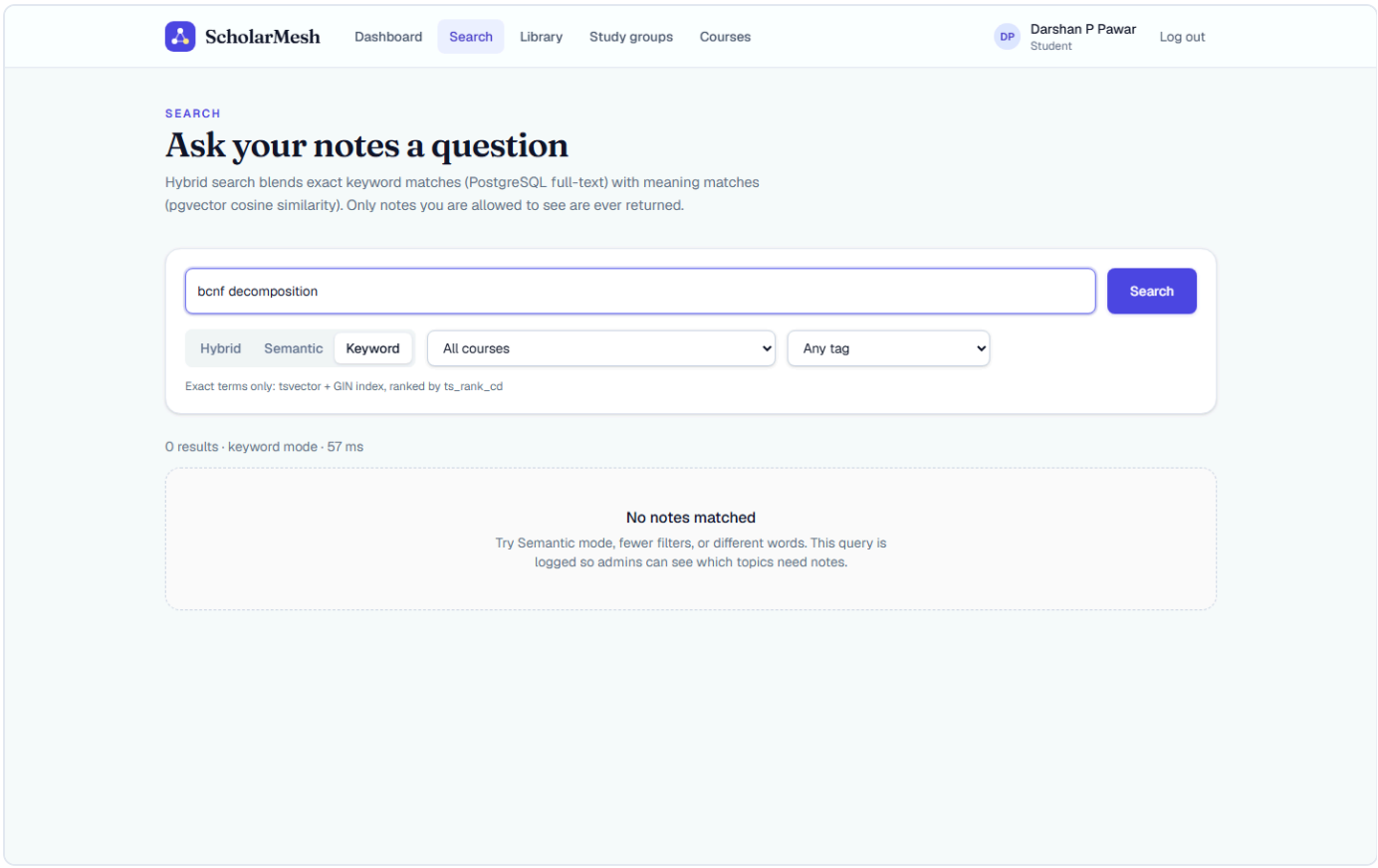


Figure 13 — Keyword mode: "bcnf decomposition" finds nothing — no note contains the word "decomposition"

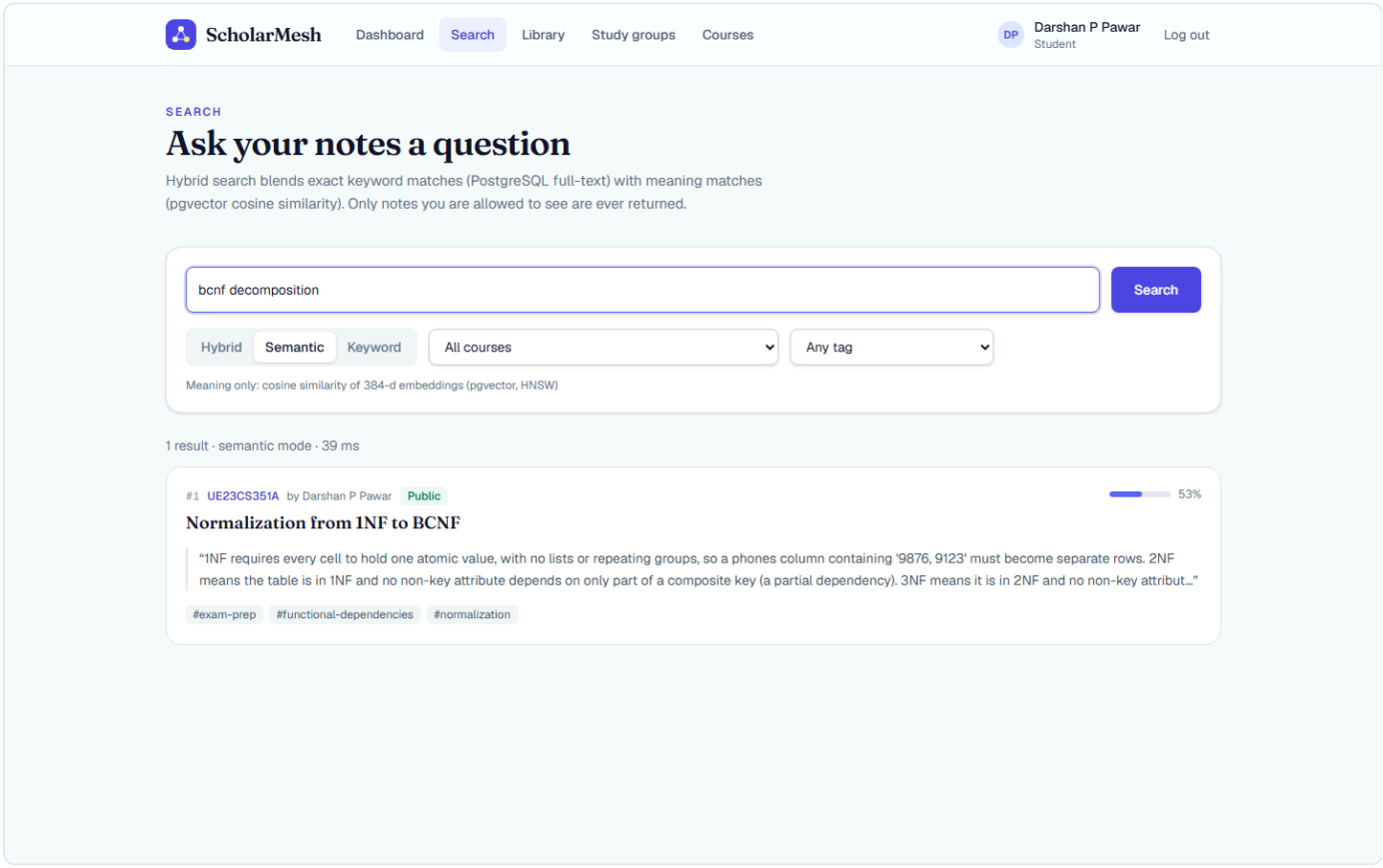


Figure 14 — Semantic mode: the same question finds the normalization note by meaning

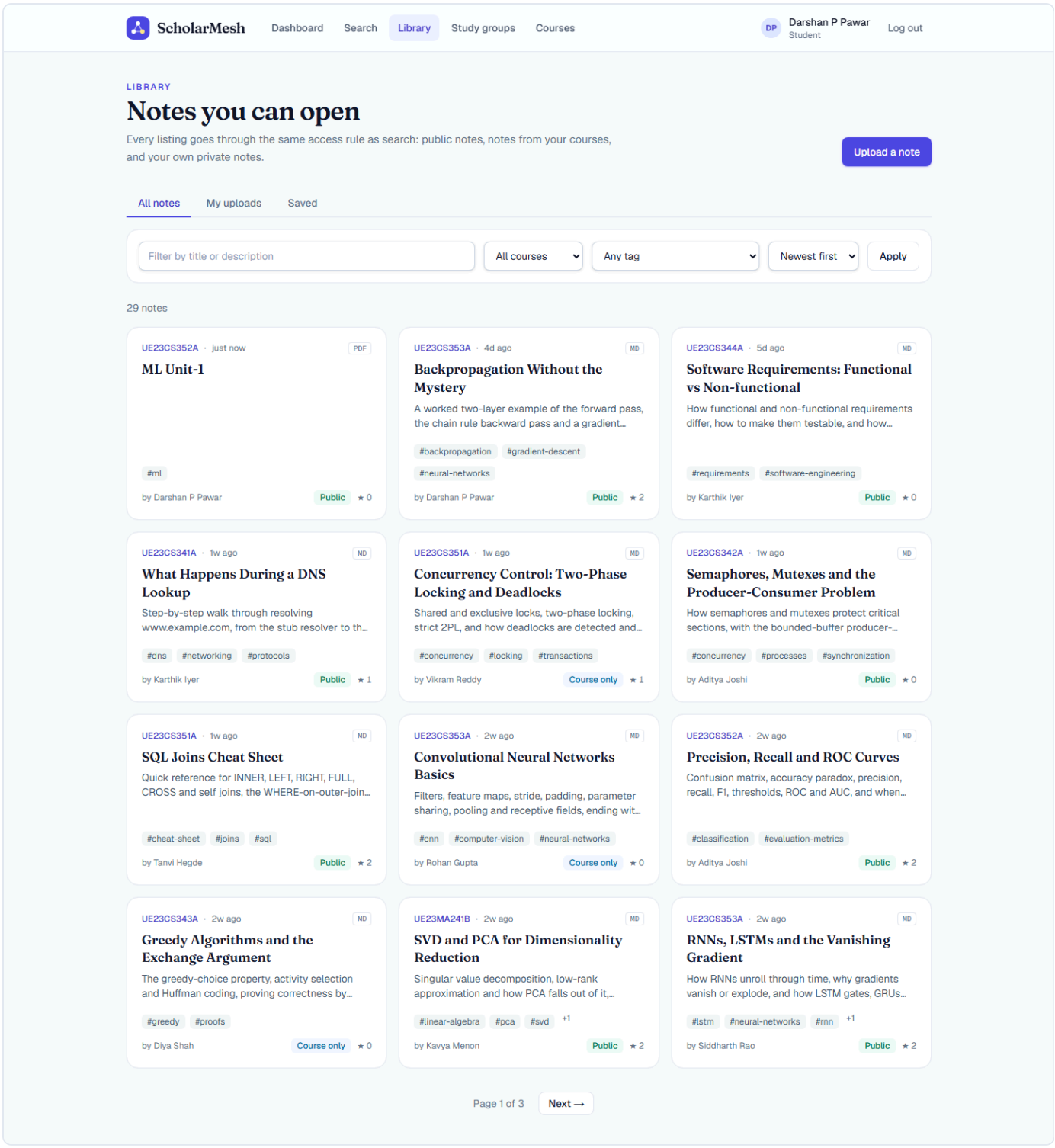


Figure 15 — Library — filters, sorting and pagination over the visible documents

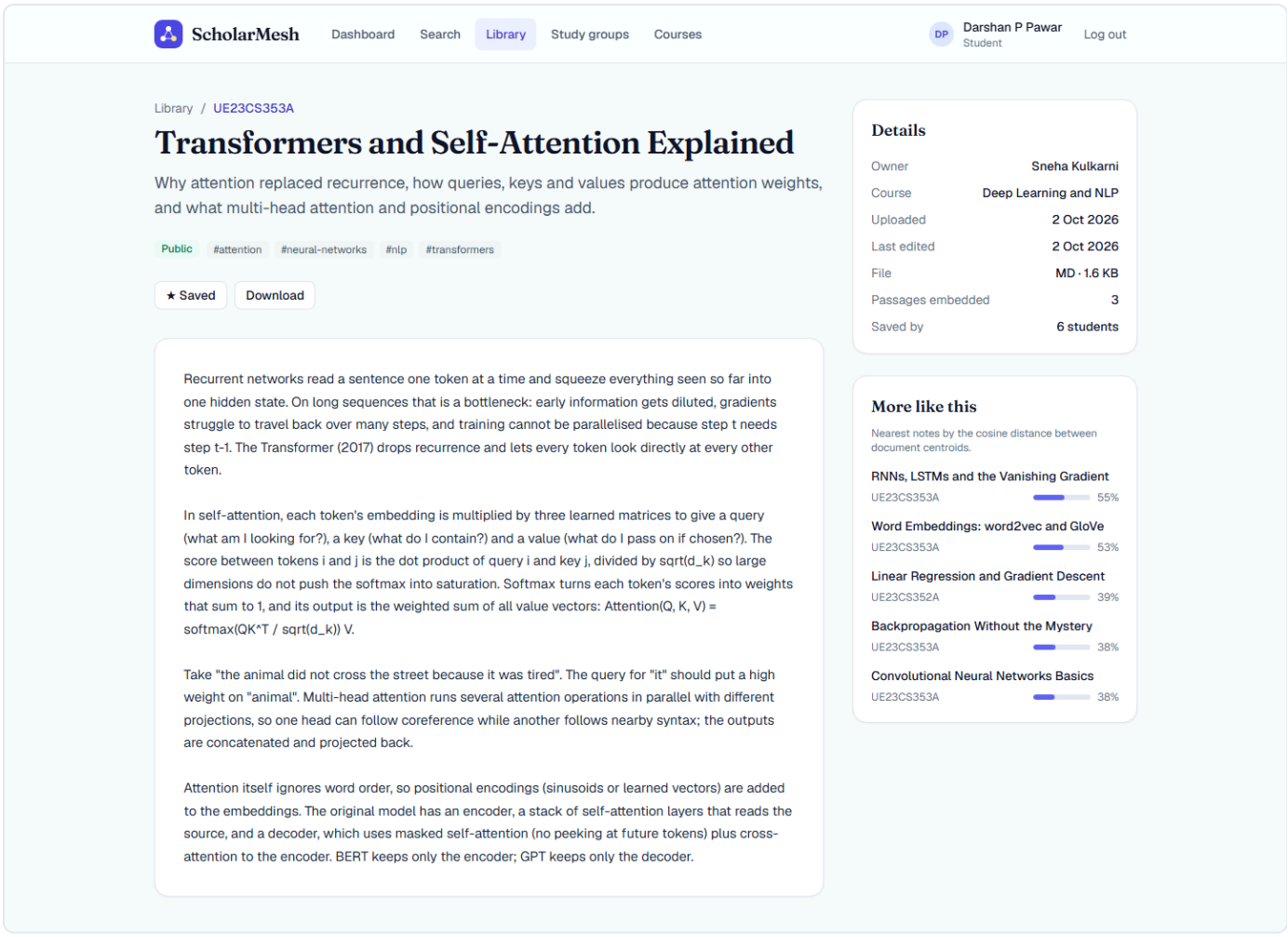


Figure 16 — Note page — metadata, passages, bookmark, download and “more like this” (centroid similarity)

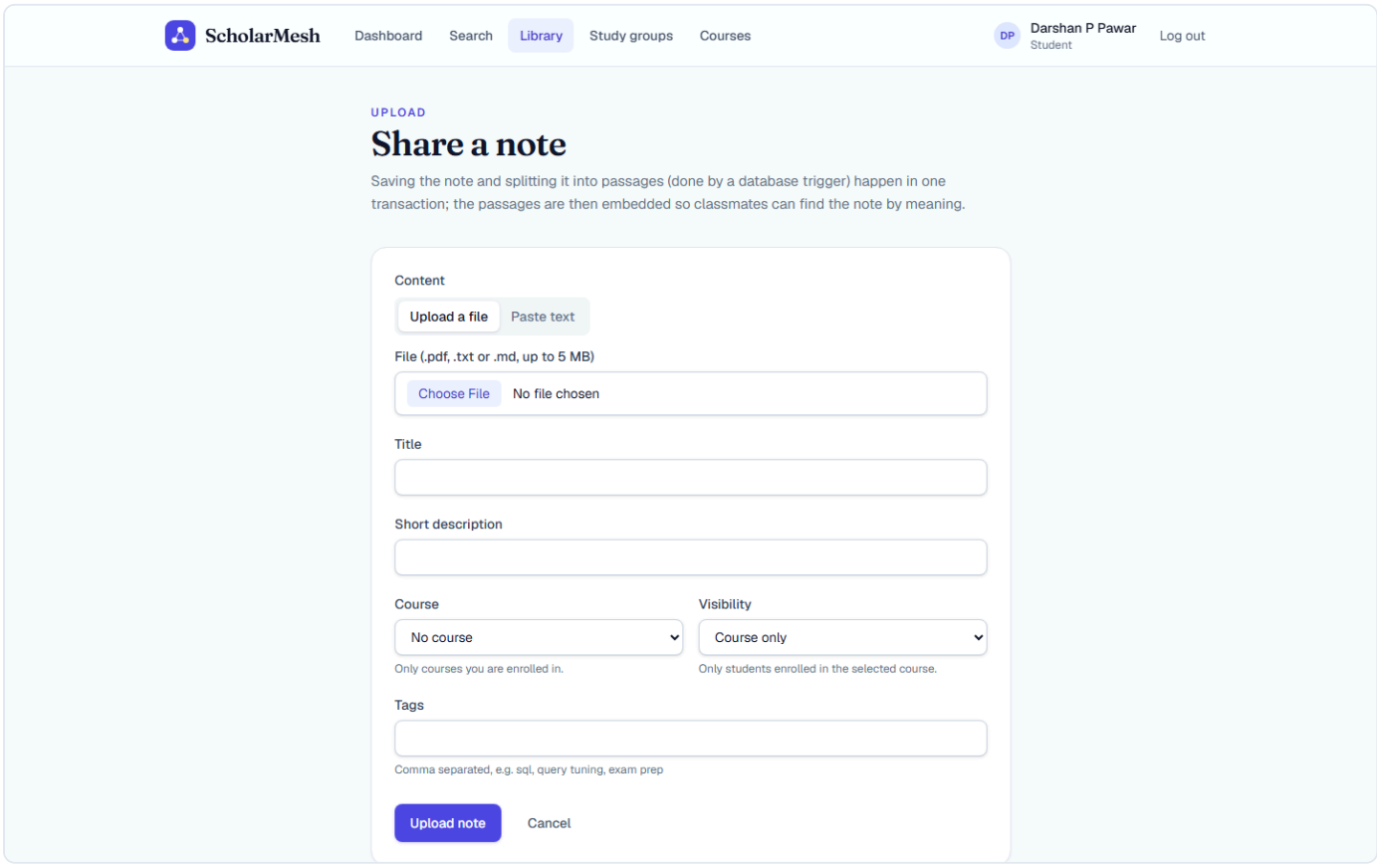


Figure 17 — Upload form — file or pasted text, course, visibility, tags

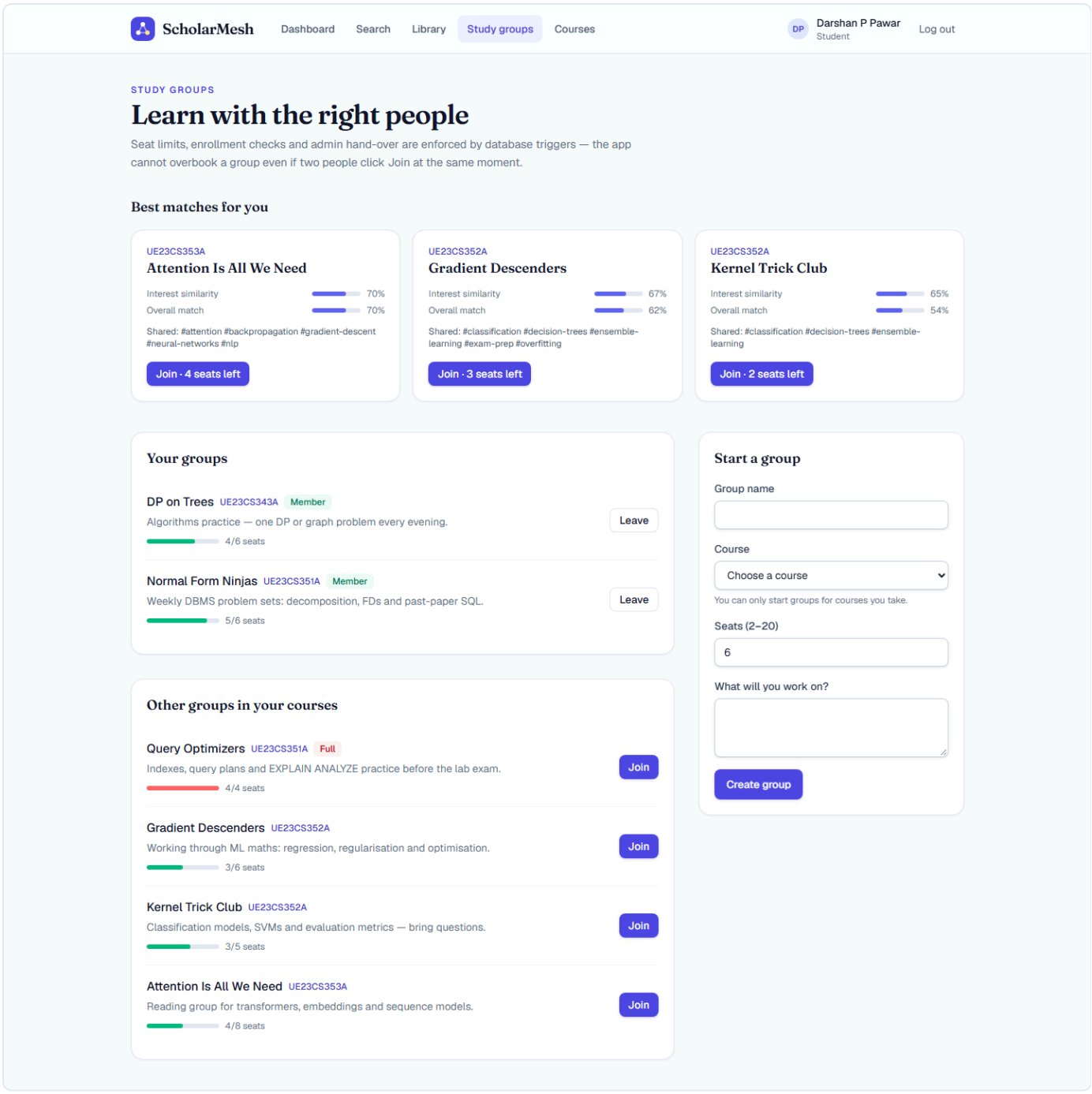


Figure 18 — Study groups — best matches, own groups, other groups with live seat counts

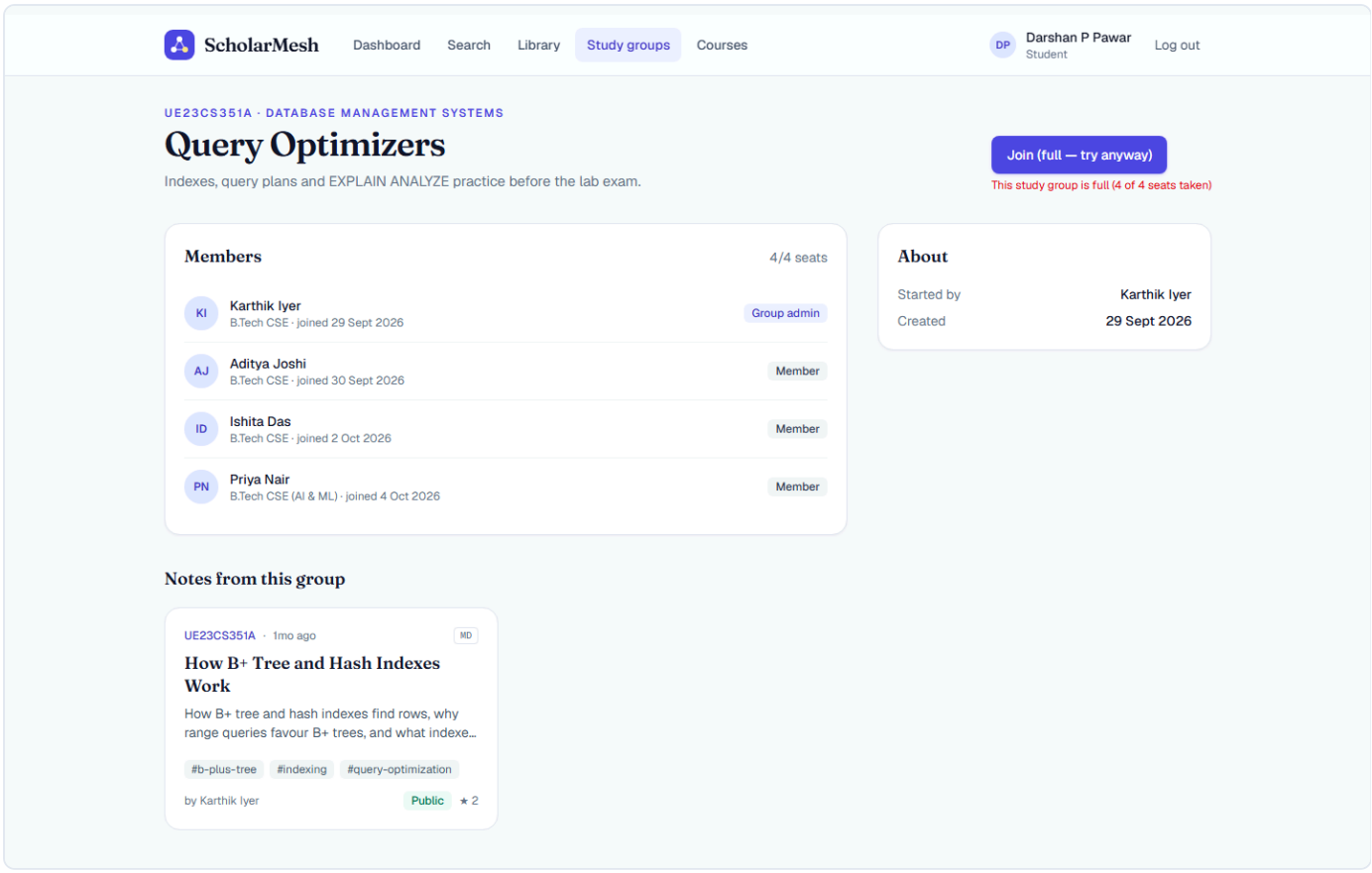


Figure 19 — Joining a full group: the database trigger refuses ("4 of 4 seats taken") and the UI shows its message

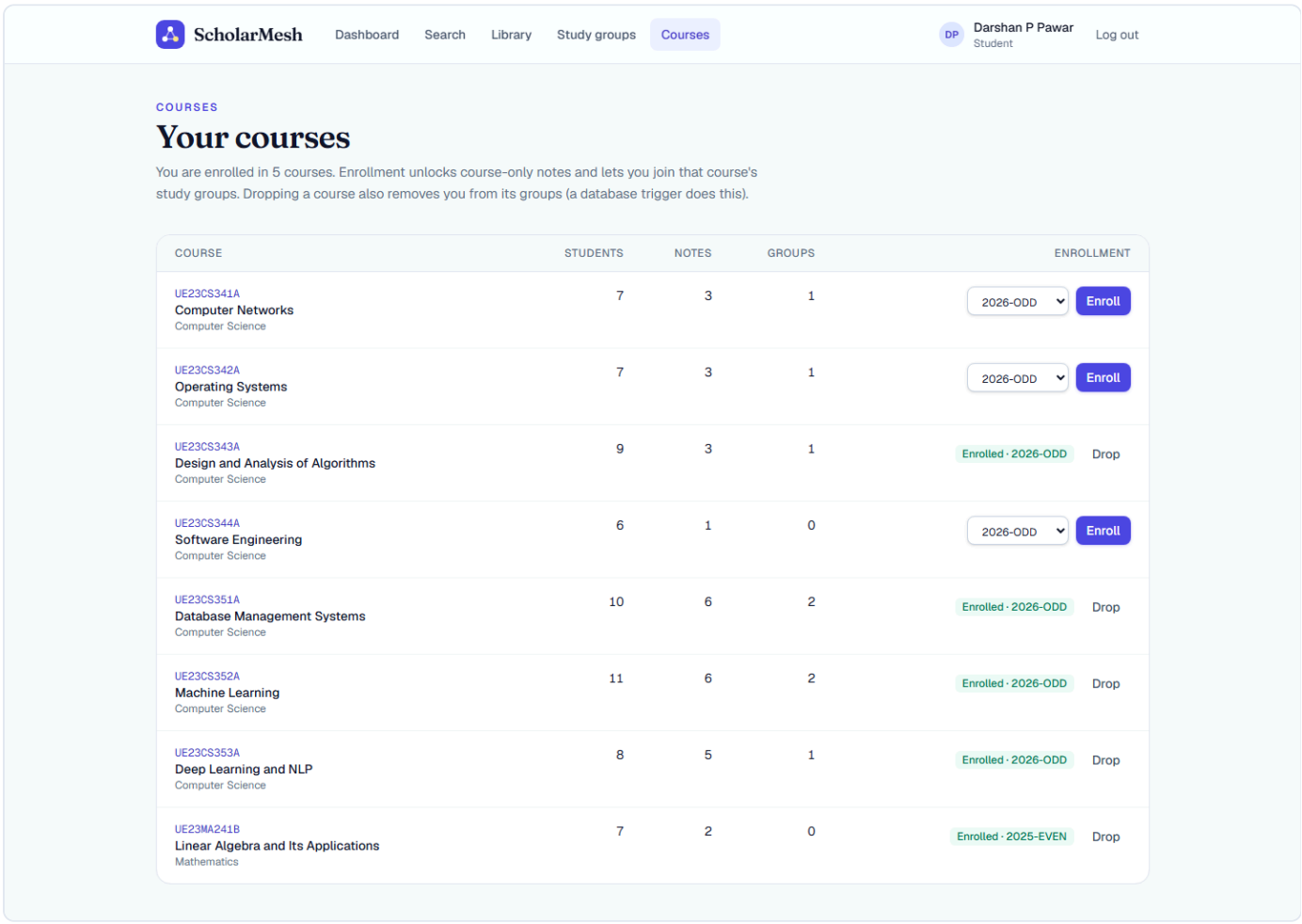


Figure 20 — Courses — enroll per semester or drop

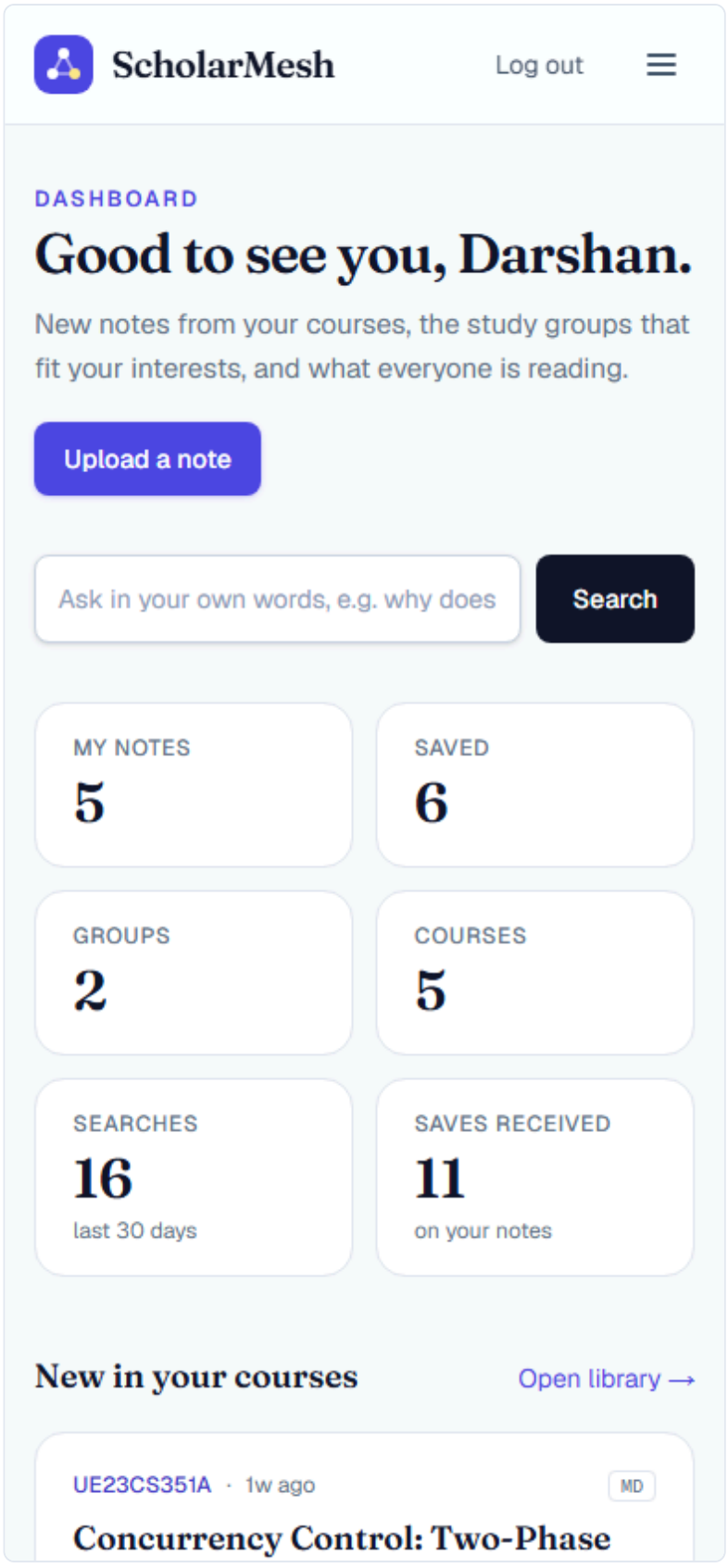


Figure 21 — Responsive layout on a 390 px phone screen

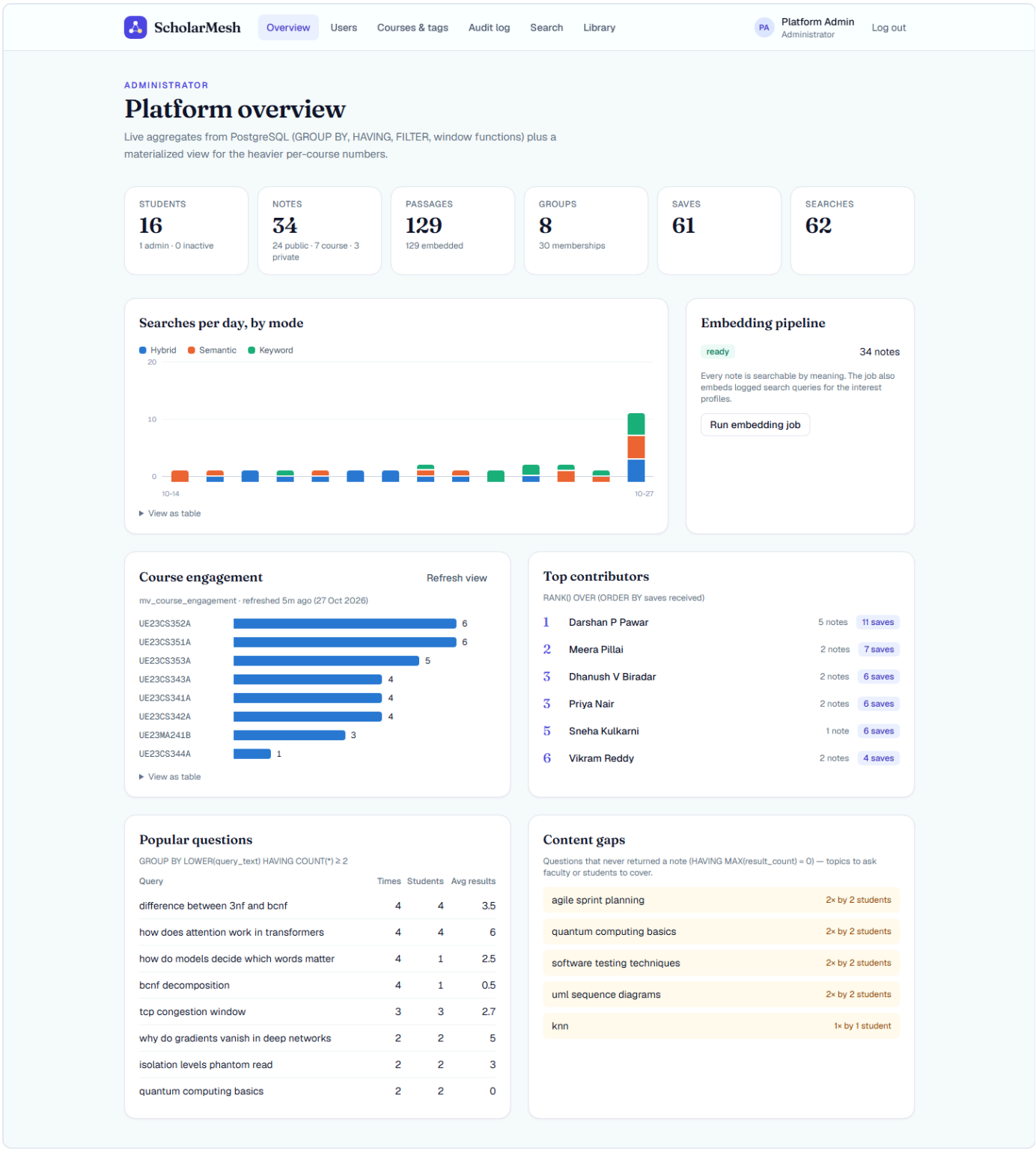


Figure 22 — Admin overview — totals, searches per day by mode, embedding pipeline, materialized-view course engagement, RANK() leaderboard, popular questions and content gaps

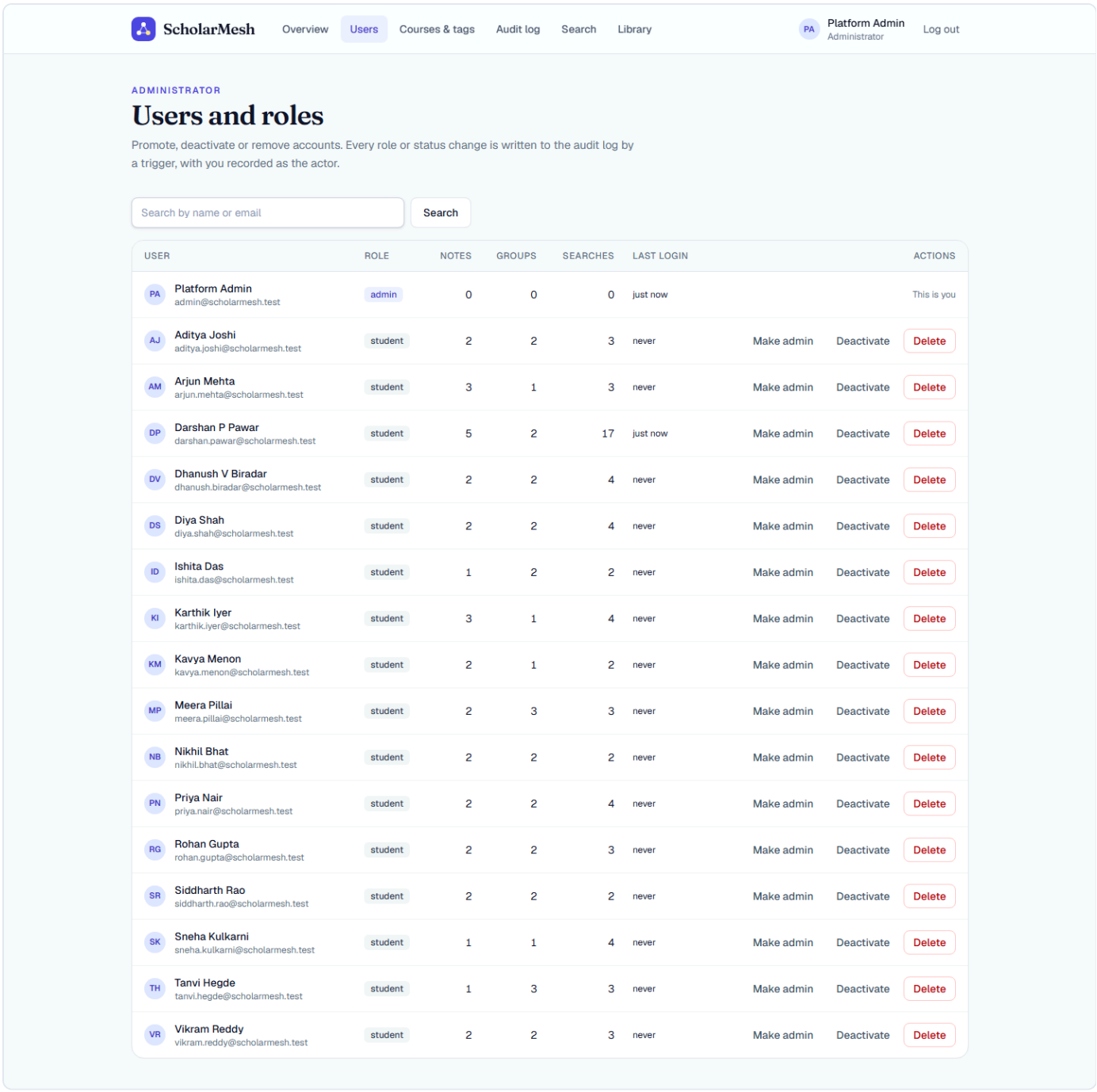


Figure 23 — Admin — users, roles and activation

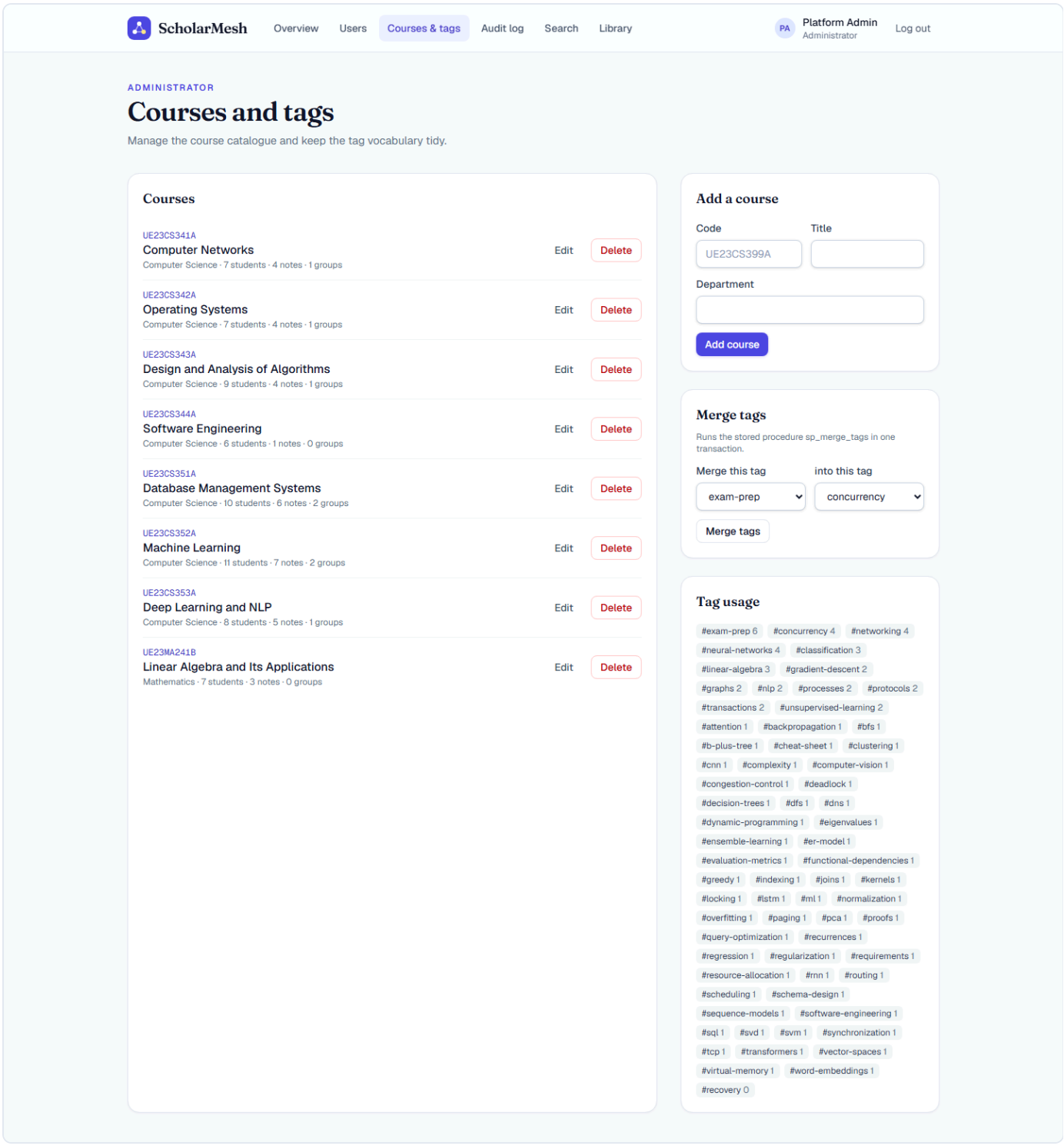


Figure 24 — Admin — course catalogue and tag merge (stored procedure)

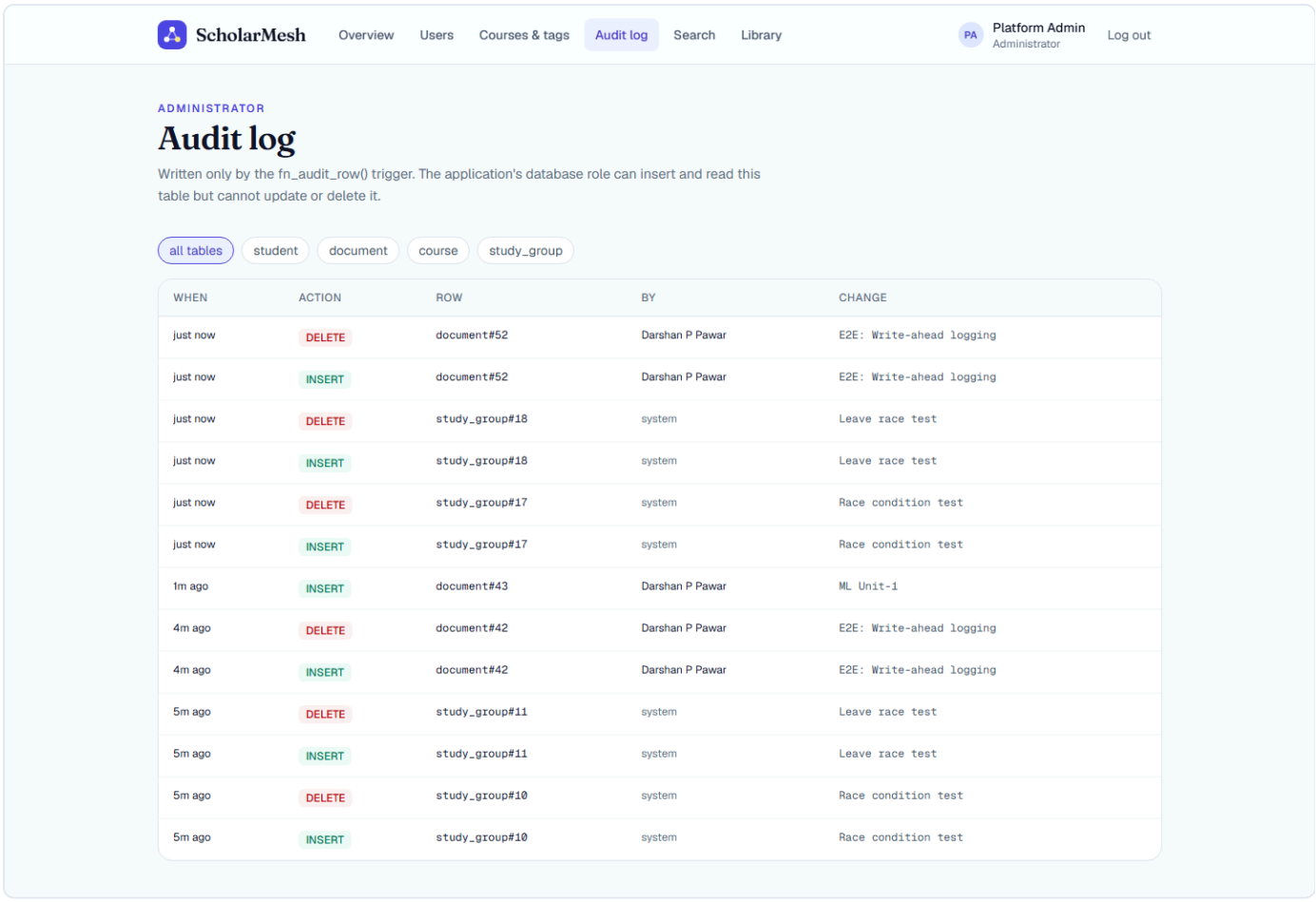


Figure 25 — Admin — audit log written by triggers, with the acting user

9. Evaluation rubrics — completion status

Every item the project brief evaluates, with its status and where the evidence is in this report.

9.1 Rubric checklist

22/23	1	101	11
items complete	pending	tests passing	advanced DB features

#	Rubric item	Status	Evidence in this report
Evaluation / demonstration (project brief)			
1	Problem statement and motivation	✓ Complete	§3.1
2	ER diagram and database design	✓ Complete	§5.2; design choices §4.1
3	Relational schema	✓ Complete	§5.3; data dictionary in Appendix A
4	Database implementation	✓ Complete	§5.4–§5.11, §6.2
5	Important queries / operations	✓ Complete	§5.12 (SQL), §6.3 (vector), with real output
6	Next.js interface	✓ Complete	§7, §8.2; demo video
7	Database integration	✓ Complete	§7.2–§7.4, §8.1
8	Advanced database features (at least 4)	✓ Complete	§5.13 — 11 features
9	Test cases	✓ Complete	§9.2 — 101 of 101 passing
10	Individual contribution of team members	✓ Complete	§9.3
Technical requirements			
11	SQL + vector database (Option 2), choice justified per kind of data	✓ Complete	§4.1
12	Vector component: data model, embeddings, indexes, retrieval strategy	✓ Complete	§6
13	DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	✓ Complete	§5.4–§5.7
14	DML: INSERT, UPDATE, DELETE, SELECT with JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	✓ Complete	§5.12
15	Backend / API layer (Next.js Route Handlers)	✓ Complete	§7.3–§7.4, §8.1
16	Authentication and at least 2 roles	✓ Complete	student / admin (+ group admin / member); §3.4, §7.5
17	Validation and integrity at database level	✓ Complete	§5.5, §5.9, §5.10
18	Security: no plain-text passwords, credentials only in environment variables	✓ Complete	§5.11, §7.5
Level 3 expectations			
19	Beyond CRUD: clear link between concepts, design, implementation and the app	✓ Complete	§3.1, §6, §8.1
20	Original problem, database design and implementation	✓ Complete	§3

#	Rubric item	Status	Evidence in this report
Deliverables			
21	Documentation: report, README, setup guide, screenshots, demo video	✓ Complete	this report; Appendix B; §8.2
22	Git + GitHub repository with README	✓ Complete	github.com/Darshanpawar7/DBMS-Mini-Project
23	Individual commits from each member's own GitHub account	Pending	Pending — all commits so far are from Darshan's account; Dhanush V Biradar to push his own commits

9.2 Test cases and results

101/101	47	44	10
tests passed	unit	database integration	end-to-end

- **Unit tests** (Vitest): input validation, password hashing (the hash satisfies the database CHECK), session tokens (expired, forged and tampered tokens rejected), error mapping, CSRF guard, rate limiter, safe highlighting.
- **Database integration tests** (Vitest + pg) run as the application role against the real database, each inside `BEGIN … ROLLBACK` with savepoints, so they can assert that a statement *fails* with the right SQLSTATE and leave no data behind. They cover constraints, every trigger, access control, the three search modes, recommendations, procedures, privileges and a real two-connection race for the last seat.
- **End-to-end tests** (Playwright, Chromium) drive the production build: login, semantic and keyword search, privacy of other students' notes, upload and delete, the full-group rule, admin-only pages.
- Also checked: TypeScript strict type-check, ESLint, production build, and the SQL demonstration script (only the intended "group is full" error).
- **Independent code and security review** of the finished code: 15 distinct findings (redirect loop for stale sessions, private-note metadata in tag counts and recommendations, a leave race, upload limits, rate-limit spoofing, open redirect, …). All were fixed and covered by new tests, except open self-enrollment, which is documented as a limitation (§3.3).

Commands: `npm test`, `npm run test:e2e`, `npm run typecheck`, `npm run lint` (in `web/`).

#	Layer	Test case	Result
1	Database integration	STUDENT constraints › rejects an invalid email (chk_student_email)	passed
2	Database integration	STUDENT constraints › rejects a plain-text password (chk_student_pwd_bcrypt)	passed
3	Database integration	STUDENT constraints › email uniqueness ignores case and whitespace (unique index + normalise trigger)	passed
4	Database integration	STUDENT constraints › role and year_of_study are restricted	passed
5	Database integration	DOCUMENT constraints › course-only visibility requires a course (chk_document_course_vis)	passed
6	Database integration	DOCUMENT constraints › file type and visibility are whitelisted	passed
7	Database integration	DOCUMENT constraints › a document must belong to an existing student (foreign key)	passed
8	Database integration	other tables › semester format, duplicate enrollment, tag format, group size	passed
9	Database integration	other tables › tag names are normalised by trigger before the CHECK runs	passed
10	Database integration	other tables › group names are unique per course	passed

#	Layer	Test case	Result
11	Database integration	access control (fn_visible_doc_ids) › another student's private note is invisible; your own is visible	passed
12	Database integration	access control (fn_visible_doc_ids) › course-only notes are visible to enrolled students only	passed
13	Database integration	access control (fn_visible_doc_ids) › admins see every document	passed
14	Database integration	access control (fn_visible_doc_ids) › search never returns a document the student cannot see	passed
15	Database integration	search functions › keyword search finds exact terms and returns highlight markers	passed
16	Database integration	search functions › semantic search ranks a document first for its own passage (similarity ≈ 1)	passed
17	Database integration	search functions › hybrid search fuses both rankings (RRF)	passed
18	Database integration	search functions › course and tag filters narrow every search	passed
19	Database integration	search functions › fn_chunk_text never exceeds the limit and keeps every word	passed
20	Database integration	recommendations › only open groups in the student's courses that they have not joined	passed
21	Database integration	recommendations › private notes never shape what other students are shown	passed
22	Database integration	recommendations › study partners share at least one course	passed
23	Database integration	stored procedures › sp_join_group adds a member and rejects a duplicate (SM409)	passed
24	Database integration	stored procedures › sp_merge_tags re-points documents and removes the old tag	passed
25	Database integration	least-privilege application role › cannot drop or alter tables	passed
26	Database integration	least-privilege application role › can append to the audit log but never edit or erase it	passed
27	Database integration	least-privilege application role › can refresh the materialized view only through the SECURITY DEFINER function	passed
28	Database integration	transactions › a failing statement rolls back the whole unit of work	passed
29	Database integration	transactions › two students racing for the last seat: exactly one gets it	passed
30	Database integration	transactions › the last two members leaving at the same moment never leave an orphan group	passed
31	Database integration	document triggers › T4 splits content into chunks and T3 marks the document pending	passed
32	Database integration	document triggers › editing content re-chunks; editing only the title does not	passed
33	Database integration	document triggers › bookkeeping updates (embedding status) do not change 'last edited'	passed
34	Database integration	document triggers › deleting a course turns its course-only notes private instead of failing	passed
35	Database integration	study-group triggers › T6 makes the creator the group admin	passed
36	Database integration	study-group triggers › T5 refuses a member who is not enrolled in the course (SM002)	passed
37	Database integration	study-group triggers › T5 refuses to overfill a group (SM001)	passed
38	Database integration	study-group triggers › T8 blocks demoting the last admin (SM003)	passed
39	Database integration	study-group triggers › T7 promotes the longest-standing member when the admin leaves	passed
40	Database integration	study-group triggers › T7 deletes a group when its last member leaves	passed
41	Database integration	study-group triggers › deleting the creator's account keeps the group (created_by SET NULL)	passed
42	Database integration	study-group triggers › T9 blocks shrinking max_members below the member count (SM004)	passed
43	Database integration	study-group triggers › T10 dropping a course also leaves its study groups	passed
44	Database integration	audit trigger (T11) › records the acting user from app.current_user_id and strips secrets	passed
45	Unit	password hashing › produces a bcrypt hash the database CHECK accepts	passed
46	Unit	password hashing › verifies the right password and rejects a wrong one	passed

#	Layer	Test case	Result
47	Unit	password hashing › hashes are salted (same input, different hash)	passed
48	Unit	session tokens › round-trips user id and role	passed
49	Unit	session tokens › rejects a token signed with another secret	passed
50	Unit	session tokens › rejects a tampered token (role escalation attempt)	passed
51	Unit	session tokens › rejects an expired token	passed
52	Unit	session tokens › rejects missing or garbage tokens and unknown roles	passed
53	Unit	toErrorResponse › ApiError keeps its status and message	passed
54	Unit	toErrorResponse › ZodError becomes 400 with per-field messages	passed
55	Unit	toErrorResponse › maps PostgreSQL error { code: '23505', …(1) }	passed
56	Unit	toErrorResponse › maps PostgreSQL error { code: '23514', …(1) }	passed
57	Unit	toErrorResponse › maps PostgreSQL error { code: '23503' }	passed
58	Unit	toErrorResponse › maps PostgreSQL error { code: 'SM001', …(1) }	passed
59	Unit	toErrorResponse › maps PostgreSQL error { code: 'SM002', …(1) }	passed
60	Unit	toErrorResponse › unknown errors become a generic 500 without internals	passed
61	Unit	request guards › cross-site mutation is blocked, same-site and GET are allowed	passed
62	Unit	request guards › parseId accepts positive integers only	passed
63	Unit	request guards › readJson rejects invalid JSON and non-objects	passed
64	Unit	format helpers › highlightParts turns ts_headline markers into safe parts (no HTML)	passed
65	Unit	format helpers › timeAgo and formatBytes	passed
66	Unit	rateLimit › allows the limit, then answers 429 until the window resets	passed
67	Unit	safeNextPath (post-login redirect) › /groups/3 -> /groups/3	passed
68	Unit	safeNextPath (post-login redirect) › /search?q=bcnf -> /search?q=bcnf	passed
69	Unit	safeNextPath (post-login redirect) › //evil.com -> /dashboard	passed
70	Unit	safeNextPath (post-login redirect) › /\evil.com -> /dashboard	passed
71	Unit	safeNextPath (post-login redirect) › /%5Cevil.com -> /%5Cevil.com	passed
72	Unit	safeNextPath (post-login redirect) › https://evil.com/login -> /dashboard	passed
73	Unit	safeNextPath (post-login redirect) › javascript:alert(1) -> /dashboard	passed
74	Unit	registerSchema › accepts a valid student and normalises the email	passed
75	Unit	registerSchema › rejects weak password short	passed
76	Unit	registerSchema › rejects weak password onlyletters	passed
77	Unit	registerSchema › rejects weak password 12345678	passed
78	Unit	registerSchema › rejects a bad email and a one-letter name	passed
79	Unit	registerSchema › rejects a year of study outside 1-6	passed
80	Unit	loginSchema › requires a password	passed
81	Unit	documentMetaSchema › turns an empty course into null	passed
82	Unit	documentMetaSchema › course-only visibility needs a course	passed
83	Unit	documentMetaSchema › rejects unknown visibility and short titles	passed
84	Unit	documentMetaSchema › caps the number of tags at 8	passed

#	Layer	Test case	Result
85	Unit	splitTags › splits, trims, lower-cases, hyphenates and de-duplicates	passed
86	Unit	splitTags › returns [] for non-strings	passed
87	Unit	other schemas › semester must look like 2026-ODD	passed
88	Unit	other schemas › group size must be 2-20	passed
89	Unit	other schemas › course codes are upper-cased and checked	passed
90	Unit	other schemas › search defaults to hybrid and needs 2+ characters	passed
91	Unit	tag normalisation › array tags with spaces are hyphenated and de-duplicated like the database trigger does	passed
92	End-to-end (Chromium)	anonymous visitors are sent to the login page	passed
93	End-to-end (Chromium)	wrong password shows an error	passed
94	End-to-end (Chromium)	admin sees the platform overview and the audit log	passed
95	End-to-end (Chromium)	student journey › dashboard shows stats, feed and recommendations	passed
96	End-to-end (Chromium)	student journey › semantic search finds a note with no shared keywords	passed
97	End-to-end (Chromium)	student journey › keyword search highlights the matched term	passed
98	End-to-end (Chromium)	student journey › another student's private note is not reachable	passed
99	End-to-end (Chromium)	student journey › upload a note by pasting text, then delete it	passed
100	End-to-end (Chromium)	student journey › a full study group refuses another member	passed
101	End-to-end (Chromium)	student journey › students cannot open admin pages	passed

9.3 Individual contributions

Both members worked on the database and on the application, as the brief requires. Each owned one end-to-end slice — **notes and search** (Darshan) and **people, groups and administration** (Dhanush) — and reviewed the other's work.

Member	Database work	Application work
Darshan P Pawar PES2UG24CS143	• Phase 1 ER diagram, schema and DDL; Phase 1 review and fixes (§5.1) • Phase 2 schema evolution (`ALTER`s, DOC_CHUNK, AUDIT_LOG, CHECK constraints, indexes) • pgvector set-up, HNSW and GIN indexes, chunking function and document triggers (T3, T4) • Keyword, semantic and hybrid search functions; similarity threshold calibration • Seed data (courses, students, 33 study notes)	• Upload pipeline (PDF/text extraction, one-transaction save, embedding worker) • Search page, library, note detail and upload pages • Database setup script and embedding job • Report, ER and architecture diagrams, screenshots
Dhanush V Biradar PES2UG24CS155	• Views and the materialized view; interest-profile view • Study-group triggers (seat limit, creator admin, succession, guards) and procedures `sp_join_group`, `sp_leave_group`, `sp_merge_tags` • Recommendation functions (groups and partners) • Audit trigger, least-privilege app role and grants • Important-queries script (§5.12, §6.3) and EXPLAIN analysis	• Authentication, sessions, roles and the proxy gate; security hardening • API layer: validation, error mapping, CSRF check, rate limiting • Dashboard, study groups, courses and all admin pages (charts, users, audit log) • Automated tests: unit, database integration and Playwright end-to-end

Repository: `https://github.com/Darshanpawar7/DBMS-Mini-Project`. It contains `db/` (SQL), `web/` (Next.js app and tests) and `docs/` (this report, diagrams, screenshots, demo video). Each member is to commit their own work from their own GitHub account, so that the history shows individual contributions (current status in §9.1). Setup and run instructions are in `README.md` and Appendix B.

9.4 Conclusion and future enhancements

ScholarMesh shows how one PostgreSQL database can serve two paradigms at once: a strictly validated relational model for people, courses, groups and activity, and a vector store for meaning-based retrieval — joined in single SQL statements so that access control, ranking and integrity are guaranteed by the database rather than by application code. Phase 1's design was corrected (redundant and missing indexes, a dangling vector pointer, a destructive cascade, missing roles and rules) and extended with chunk-level embeddings, full-text search, views, a materialized view, stored functions and procedures, triggers, an audit trail and a least-privilege application role. A Next.js application exposes all of it through a validated API, and 101 automated tests confirm the behaviour, including real races for the last seat in a group and for leaving one.

Future enhancements

- **OCR** for scanned PDFs and handwritten notes; image and diagram embeddings.
- **Verified enrollment** — sync `ENROLLMENT` from the university system, or require a per-course join code.
- **Row-Level Security** policies so the access rule is enforced even for ad-hoc SQL, not only through the functions.

- **Background job queue** (e.g. a worker table polled with `SELECT … FOR UPDATE SKIP LOCKED`) for large uploads.
- **Re-ranking** the top hybrid results with a cross-encoder; per-course embedding fine-tuning.
- **Notifications** when a recommended group has a free seat; meeting scheduling inside groups.
- **Table partitioning** of `SEARCH_LOG` and `AUDIT_LOG` by month as they grow; a Redis-backed rate limiter for multiple servers.

Appendix A — Data dictionary

Generated from the PostgreSQL catalog (`information_schema` , `pg_constraint` and the `COMMENT ON` descriptions stored with the schema) by `docs/report/data-dictionary.sql` .

STUDENT

Registered user. role = student | admin (platform moderator).

Column	Type	Null	Default	Key	Description
`student_id`	integer	NOT NULL	serial	PK	Surrogate key.
`full_name`	varchar(100)	NOT NULL			Display name; trimmed by trigger.
`email`	varchar(120)	NOT NULL			Login id; lower-cased by trigger, unique case-insensitively.
`password_hash`	varchar(255)	NOT NULL			bcrypt hash (cost 10); CHECK rejects anything else.
`program`	varchar(80)	null			Degree programme, e.g. B.Tech CSE.
`year_of_study`	smallint	null			Year 1-6.
`created_at`	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Registration time.
`role`	varchar(10)	NOT NULL	'student'		System role: student or admin.
`is_active`	boolean	NOT NULL	true		Admins can deactivate an account without deleting its data.
`last_login_at`	timestamptz	null			Updated on every successful login.

- `chk_student_email` : `CHECK (((email)::text ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'::text))`
- `chk_student_name` : `CHECK ((length(btrim((full_name)::text)) >= 2))`
- `chk_student_pwd_bcrypt` : `CHECK (((password_hash)::text ~ '^\$2[aby]\$[0-9]{2}\$[./A-Za-z0-9]{53}$'::text))`
- `chk_student_role` : `CHECK (((role)::text = ANY ((ARRAY['student'::character varying, 'admin'::character varying])::text[])))`
- `student_year_of_study_check` : `CHECK (((year_of_study >= 1) AND (year_of_study <= 6)))`

COURSE

A course offered in the programme; scopes documents, enrollments and groups.

Column	Type	Null	Default	Key	Description
`course_id`	integer	NOT NULL	serial	PK	Surrogate key.
`course_code`	varchar(20)	NOT NULL		UQ	University course code, e.g. UE23CS351A.
`course_title`	varchar(150)	NOT NULL			Course name.
`department`	varchar(80)	null			Offering department.

- `chk_course_code` : `CHECK (((course_code)::text ~ '^[A-Z0-9-]{3,20}$'::text))`
- `chk_course_title` : `CHECK ((length(btrim((course_title)::text)) >= 3))`

DOCUMENT

An uploaded note/paper with its extracted text and access level.

Column	Type	Null	Default	Key	Description
`doc_id`	integer	NOT NULL	serial	PK	Surrogate key.

Column	Type	Null	Default	Key	Description
title	varchar(200)	NOT NULL			Title shown in listings (search weight A).
file_path	varchar(255)	NOT NULL			Random stored file name (never the client name).
file_type	varchar(20)	NOT NULL			pdf | txt | md.
visibility	varchar(10)	NOT NULL	'private'		public = everyone, course = enrolled students, private = owner only.
upload_date	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Upload time.
student_id	integer	NOT NULL		FK → STUDENT	Owner (UPLOADS).
course_id	integer	null		FK → COURSE	Course the note belongs to (BELONGS_TO); optional.
description	varchar(500)	null			One-line summary (search weight B).
content	text	null			Plain text extracted from the uploaded file.
file_size_bytes	integer	null			Size of the uploaded file, 1 B - 10 MB.
embedding_status	varchar(10)	NOT NULL	'pending'		pending until every chunk has an embedding, then ready.
updated_at	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Last edit, stamped by trigger.
search_tsv	tsvector	null	generated		Generated weighted tsvector: title (A), description (B), content (C).

- chk_document_course_vis: `CHECK ((((visibility)::text <> 'course'::text) OR (course_id IS NOT NULL)))`
- chk_document_embedding: `CHECK (((embedding_status)::text = ANY ((ARRAY['pending'::character varying, 'ready'::character varying, 'failed'::character varying])::text[])))`
- chk_document_file_size: `CHECK (((file_size_bytes IS NULL) OR ((file_size_bytes >= 1) AND (file_size_bytes <= 10485760))))`
- chk_document_file_type: `CHECK (((file_type)::text = ANY ((ARRAY['pdf'::character varying, 'txt'::character varying, 'md'::character varying])::text[])))`
- chk_document_title: `CHECK ((length(btrim((title)::text)) >= 3))`
- document_visibility_check: `CHECK (((visibility)::text = ANY ((ARRAY['public'::character varying, 'course'::character varying, 'private'::character varying])::text[])))`

TAG

Normalised topic label (lower-case, hyphenated).

Column	Type	Null	Default	Key	Description
tag_id	integer	NOT NULL	serial	PK	Surrogate key.
tag_name	varchar(50)	NOT NULL		UQ	Normalised label, e.g. query-optimization.

- chk_tag_name: `CHECK (((tag_name)::text ~ '^[a-z0-9][a-z0-9+#.-]{0,49}$'::text))`

STUDY_GROUP

A study group run for one course, capped at max_members.

Column	Type	Null	Default	Key	Description
group_id	integer	NOT NULL	serial	PK	Surrogate key.
group_name	varchar(100)	NOT NULL		UQ	Unique within its course.
max_members	smallint	NOT NULL	6		Seat limit 2-20, enforced by trigger on join.
created_at	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Creation time.
course_id	integer	NOT NULL		FK → COURSE, UQ	Course the group is run for (RUN_FOR).
created_by	integer	null		FK → STUDENT	Creator; NULL if the account was deleted (group survives).
description	varchar(300)	null			What the group works on.

- chk_group_max_members: CHECK (((max_members >= 2) AND (max_members <= 20)))
- chk_group_name: CHECK ((length(btrim((group_name)::text)) >= 3))

SEARCH_LOG

Every search a student runs; also an interest signal for matching.

Column	Type	Null	Default	Key	Description
log_id	integer	NOT NULL	serial	PK	Surrogate key.
query_text	varchar(255)	NOT NULL			What the student typed.
searched_at	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Search time.
student_id	integer	NOT NULL		FK → STUDENT	Who searched (SEARCHES).
search_mode	varchar(10)	NOT NULL	'hybrid'		keyword | semantic | hybrid.
result_count	integer	null			Hits returned; 0 marks a content gap.
query_embedding	vector(384)	null			Embedding of the query text; feeds the interest profile.

- chk_search_count: CHECK (((result_count IS NULL) OR (result_count >= 0)))
- chk_search_mode: CHECK (((search_mode)::text = ANY ((ARRAY['keyword'::character varying, 'semantic'::character varying, 'hybrid'::character varying])::text[])))
- chk_search_query: CHECK ((length(btrim((query_text)::text)) >= 2))

ENROLLMENT

M:N STUDENT-COURSE per semester.

Column	Type	Null	Default	Key	Description
student_id	integer	NOT NULL		FK → STUDENT, PK	Enrolled student.
course_id	integer	NOT NULL		FK → COURSE, PK	Course taken.
semester	varchar(20)	NOT NULL		PK	Format YYYY-ODD or YYYY-EVEN.

- chk_enrollment_semester: CHECK (((semester)::text ~ '^[0-9]{4}-(ODD|EVEN)$'::text))

DOCUMENT_TAG

M:N DOCUMENT-TAG.

Column	Type	Null	Default	Key	Description
doc_id	integer	NOT NULL		FK → DOCUMENT, PK	Tagged document.
tag_id	integer	NOT NULL		FK → TAG, PK	Applied tag.

BOOKMARK

M:N STUDENT-DOCUMENT saved items.

Column	Type	Null	Default	Key	Description
student_id	integer	NOT NULL		FK → STUDENT, PK	Who saved it.
doc_id	integer	NOT NULL		FK → DOCUMENT, PK	Saved document.
bookmarked_at	timestamptz	NOT NULL	CURRENT_TIMESTAMP		When it was saved.

GROUP_MEMBERSHIP

M:N STUDENT-STUDY_GROUP with a per-group role.

Column	Type	Null	Default	Key	Description
group_id	integer	NOT NULL		FK → STUDY_GROUP, PK	Group joined.
student_id	integer	NOT NULL		FK → STUDENT, PK	Member.
role	varchar(10)	NOT NULL	'member'		admin | member (per group).
joined_at	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Join time; oldest member inherits admin.

- group_membership_role_check : CHECK (((role)::text = ANY ((ARRAY['admin'::character varying, 'member'::character varying])::text[])))

DOC_CHUNK

Weak entity: ~800-char passage of a document plus its 384-d embedding.

Column	Type	Null	Default	Key	Description
chunk_id	integer	NOT NULL	serial	PK	Surrogate key (partial key is chunk_index).
doc_id	integer	NOT NULL		FK → DOCUMENT, UQ	Owning document (identifying FK).
chunk_index	integer	NOT NULL		UQ	Position of the passage inside its document.
chunk_text	text	NOT NULL			Passage text, at most 800 characters.
embedding	vector(384)	null			384-d all-MiniLM-L6-v2 sentence embedding, L2-normalised.
embedded_at	timestamptz	null			When the vector was computed.

- chk_chunk_embedded : CHECK (((embedding IS NULL) = (embedded_at IS NULL)))
- doc_chunk_chunk_index_check : CHECK ((chunk_index >= 0))
- doc_chunk_chunk_text_check : CHECK ((length(chunk_text) > 0))

AUDIT_LOG

Append-only change history written by triggers.

Column	Type	Null	Default	Key	Description
`audit_id`	bigint	NOT NULL	serial	PK	Surrogate key.
`table_name`	varchar(40)	NOT NULL			Table that changed.
`action`	varchar(10)	NOT NULL			INSERT | UPDATE | DELETE.
`row_pk`	varchar(40)	NOT NULL			Primary key of the changed row.
`actor_id`	integer	null		FK → STUDENT	User who made the change, read from the app.current_user_id setting.
`old_data`	jsonb	null			Row before the change (JSONB, secrets removed).
`new_data`	jsonb	null			Row after the change (JSONB, secrets removed).
`logged_at`	timestamptz	NOT NULL	CURRENT_TIMESTAMP		Change time.

- `audit_log_action_check` : `CHECK (((action)::text = ANY ((ARRAY['INSERT'::character varying, 'UPDATE'::character varying, 'DELETE'::character varying])::text[])))`

Appendix B — Running the project

```
cp .env.example .env                 # choose database passwords
docker compose up -d                 # PostgreSQL 17 + pgvector on port 5433
cd web && cp .env.example .env.local # DATABASE_URL (app password) + AUTH_SECRET
npm install
npm run db:setup                     # schema, functions, triggers, grants, seed data
npm run db:embed                     # compute embeddings (downloads the 23 MB model once)
npm run dev                          # http://localhost:3000
```

Demo logins: `darshan.pawar@scholarmesh.test` and `dhanush.biradar@scholarmesh.test` (students, password `Scholar@123`); `admin@scholarmesh.test / Admin@12345` (admin).